

Everything

Python Object Model & Memory Internals — Deep Dive

1. Everything is an Object

Core Idea:

In Python, **everything** that exists in memory is an object — every value, every function, every class, even the code itself.

```
# Proof that everything is an object
print(type(42))          # <class 'int'>
print(type(int))         # <class 'type'>
print(type(type))        # <class 'type'> ← type is its own type!
print(type(None))        # <class 'NoneType'>
print(type(lambda: None)) # <class 'function'>
print(type(list))        # <class 'type'>

# Even code is an object
def foo(): pass
print(type(foo.__code__)) # <class 'code'>
```

Every Object has 3 Properties:

```
x = 42

id(x)    # WHERE → memory address (unique identifier)
type(x)  # WHAT  → what kind of object
x        # VALUE → what it holds
```

Objects are First Class:

First class means you can:

```
# 1. Assign to variable
f = print

# 2. Pass as argument
```

```
def run(func):
    func("hello")
run(print)          # → hello

# 3. Return from function
def get_func():
    return print
g = get_func()
g("world")          # → world

# 4. Store in data structure
funcs = [print, len, type]
funcs[0]("hi")      # → hi
```

Interview Scenarios:

Q1: Is a class an object in Python?

```
class Dog:
    pass

print(type(Dog))      # <class 'type'> ← YES, class is an object!
print(isinstance(Dog, object)) # True
print(id(Dog))        # has memory address – definitely an object
```

Q2: What is the base of ALL objects?

```
print(int.__bases__)   # (<class 'object'>,)
print(str.__bases__)   # (<class 'object'>,)
print(list.__bases__)  # (<class 'object'>,)

# object is the root of everything
print(isinstance(42, object)) # True
print(isinstance("hi", object)) # True
print(isinstance(print, object)) # True
```

Q3: Can functions be stored and passed like data?

```
operations = {
    'add': lambda a,b: a+b,
    'sub': lambda a,b: a-b,
    'mul': lambda a,b: a*b,
```

```

}

def calculate(op, a, b):
    return operations[op](a, b)

print(calculate('add', 5, 3))    # 8
print(calculate('mul', 4, 2))    # 8
# Functions stored in dict – treated as pure data

```

Q4: What is `type(type)` ?

```

print(type(type))    # <class 'type'>
# type is its own metaclass – it created itself!
# This is the bedrock of Python's object system

```

2. Object Identity, Type, Value

Deep Dive:

```
a = [1, 2, 3]
```

LIST OBJECT	
id	= 140234567 (address)
type	= <class 'list'>
value	= [1, 2, 3]
refcount	= 1

↑
a (just a label pointing here)

Variables are just LABELS:

```

a = [1, 2, 3]
b = a           # b is another label for SAME object
c = [1, 2, 3]   # c is a label for DIFFERENT object

```

Address 1000: [1,2,3] ← a
← b

Address 2000: [1,2,3] ← c

Q1: What is the difference between `is` and `==` ?

Q2: When does `==` and `is` give different results?

Everything

```

def __init__(self, val):
    self.val = val
def __eq__(self, other):
    return self.val == other.val

b1 = Box(5)
b2 = Box(5)
print(b1 == b2)    # True  (__eq__ says same value)
print(b1 is b2)    # False (different objects in memory)

```

What is a Valid Identifier?

python

```

# Valid identifiers – only these characters:
# Letters (a-z, A-Z)
# Digits (0-9) – but NOT as first character
# Underscore (_)

hello      ✓ valid identifier
hello_world ✓ valid identifier
python3    ✓ valid identifier

hello world ✗ has SPACE    → not identifier
hello-world ✗ has HYPHEN   → not identifier
hello!     ✗ has !         → not identifier

```

Q3: Tricky — what happens when you reassign?

```

a = 5
b = a
print(a is b)    # True (both point to same int object 5)

a = 6             # a now points to NEW object 6
print(a is b)    # False (a moved, b still at 5)
print(b)         # 5 ← b unaffected

```

Q4: Can two objects have same id at different times?

```

a = [1, 2, 3]
print(id(a))    # 140234567

del a           # object destroyed, memory freed

b = [4, 5, 6]
print(id(b))    # 140234567 ← SAME address! (memory reused)

```

```
# This is why you should never store id() and compare later
# id is only unique for LIFETIME of an object
```

Q5: **None** identity trap

```
def foo(x=None):
    if x == None:    # ❌ works but wrong – could match custom __eq__
        pass
    if x is None:    # ✅ correct – checks identity
        pass

# Why is None special?
print(id(None))
a = None
b = None
print(a is b)      # Always True – None is a SINGLETON
                   # only ONE None object exists in Python
```

3. Datatypes & Memory Behavior

Immutable Types:

```
int, float, str, tuple, bool, frozenset, bytes
```

```
x = "hello"
print(id(x))          # 140001

x += " world"
print(id(x))          # 140999 ← NEW object created
                      # 140999 ← different address!
```

Mutable Types:

```
list, dict, set, bytearray
```

```
x = [1, 2, 3]
print(id(x))          # 140001

x.append(4)
print(id(x))          # 140001 ← SAME object modified
                      # 140001 ← same address!
```

Memory Layout Visualization:

```
IMMUTABLE (str):
x = "hello"          x → ["hello"] at addr 1000

x += " world"        x → ["hello world"] at addr 2000
                      addr 1000 still exists until refcount=0

MUTABLE (list):
x = [1,2,3]          x → [1,2,3] at addr 1000

x.append(4)           x → [1,2,3,4] at addr 1000 (same!)
```

Interview Scenarios:

Q1: Why are strings immutable?

```
# Safety in hashing – strings used as dict keys
d = {"hello": 1}
# If strings were mutable, key could change → dict broken!

# Thread safety – immutable objects safe across threads
# Memory efficiency – interning only works if immutable

s = "hello"
# s[0] = "H"    # ❌ TypeError – cannot modify
new_s = "H" + s[1:] # ✅ creates new string
```

Q2: Shallow vs Deep copy with mutables

```
import copy

original = [[1, 2], [3, 4]]

shallow = original.copy()    # or list(original)
deep    = copy.deepcopy(original)

original[0].append(99)

print(original)    # [[1, 2, 99], [3, 4]]
```

```
print(shallow)    # [[1, 2, 99], [3, 4]] ← affected! (shared inner lists)
print(deep)       # [[1, 2], [3, 4]]    ← unaffected! (fully independent)
```

Shallow copy:

```
original → [ ptr1, ptr2 ]
shallow  → [ ptr1, ptr2 ] ← same inner objects!
           ↓           ↓
          [1,2]    [3,4]
```

Deep copy:

```
original → [ ptr1, ptr2 ]
deep     → [ ptr3, ptr4 ] ← brand new inner objects!
           ↓           ↓
          [1,2]    [3,4]
```

Q3: Tuple with mutable inside

```
t = (1, 2, [3, 4])
# t[2] = [99]    # ❌ TypeError – tuple itself immutable
t[2].append(99)  # ✅ list inside CAN change!
print(t)         # (1, 2, [3, 4, 99])

# Tuple cannot be hashed if it contains mutable objects
# hash(t)        # ❌ TypeError: unhashable type: 'list'
hash((1, 2, 3))  # ✅ works – all immutable inside
```

Q4: Dict key rules

```
# Only hashable (immutable) types as keys
d = {}
d[(1,2,3)] = "tuple key"    # ✅
d["string"] = "string key"  # ✅
d[42] = "int key"           # ✅
# d[[1,2]] = "list key"     # ❌ TypeError: unhashable type: 'list'
# d[{1,2}] = "set key"      # ❌ TypeError: unhashable type: 'set'
```

Q5: Mutable default argument trap

```
# ❌ Classic bug
def append_to(item, lst=[]):
    lst.append(item)
    return lst
```



```

print(append_to(1))    # [1]
print(append_to(2))    # [1, 2] ← WRONG!
print(append_to(3))    # [1, 2, 3] ← getting worse!
# lst=[] created ONCE when function defined – shared across calls!

# ✅ Fix
def append_to(item, lst=None):
    if lst is None:
        lst = []        # fresh list every call
    lst.append(item)
    return lst

print(append_to(1))    # [1]
print(append_to(2))    # [2] ← correct!

```

4. Reference Counting — Deep Dive

How it works:

```

import sys

x = [1, 2, 3]
print(sys.getrefcount(x))    # 2 (x + getrefcount arg)

```

`getrefcount` itself adds 1 temporarily — so always subtract 1 from result.

Every action that changes refcount:

```

a = [1,2,3]        # refcount = 1  (a)
b = a              # refcount = 2  (a, b)
c = a              # refcount = 3  (a, b, c)
lst = [a]          # refcount = 4  (a, b, c, lst[0])
del b              # refcount = 3
lst.pop()          # refcount = 2
c = None           # refcount = 1  (c no longer points to [1,2,3])
del a              # refcount = 0  → DESTROYED 💥

```

Interview Scenarios:

Q1: What happens when refcount hits 0?

```
class Bomb:
    def __del__(self):
        print("💣 Destroyed!")

b = Bomb()      # refcount = 1
b = None        # refcount = 0 → "💣 Destroyed!" immediately!
```

Q2: Refcount in function calls

```
import sys

x = [1, 2, 3]
print(sys.getrefcount(x))    # 2

def show(obj):
    # obj is another reference inside function
    print(sys.getrefcount(obj)) # 3 (x, show arg, getrefcount)

show(x)
print(sys.getrefcount(x))    # 2 again (function call done)
```

Q3: Why refcount alone is not enough

```
# Cyclic reference – refcount NEVER hits 0
class Node:
    pass

a = Node()
b = Node()
a.next = b    # a references b
b.prev = a    # b references a

del a         # a's refcount: 2→1 (b.prev still points to it)
del b         # b's refcount: 2→1 (a.next still points to it)

# Both have refcount=1 but unreachable!
# Memory leaked without GC!
```

Q4: How to check refcount manually

```

import sys
import ctypes

x = [1, 2, 3]
addr = id(x)

# Check refcount using ctypes (advanced)
print(ctypes.c_long.from_address(addr).value) # actual refcount

# Safer way
print(sys.getrefcount(x) - 1) # subtract 1 for getrefcount itself

```

5. Garbage Collection — Deep Dive

Three Generations:

```

Generation 0 ← new objects (collected most often)
Generation 1 ← survived one GC
Generation 2 ← survived multiple GCs (collected rarely)

```

```

import gc

print(gc.get_threshold()) # (700, 10, 10)
# Gen 0 collected after 700 allocations
# Gen 1 collected after Gen 0 collected 10 times
# Gen 2 collected after Gen 1 collected 10 times

```

Interview Scenarios:

Q1: How to interact with GC

```

import gc

gc.disable() # turn off automatic GC
gc.enable()  # turn on
gc.collect() # manually run GC
gc.collect(0) # collect only generation 0
gc.collect(1) # collect only generation 1
gc.collect(2) # collect all generations

```

```
print(gc.get_count())      # (allocations since last GC per gen)
print(gc.get_threshold())  # thresholds for each generation
```

Q2: Finding memory leaks

```
import gc

gc.collect()
print(f"Garbage: {gc.garbage}")  # uncollectable objects

# Enable debug
gc.set_debug(gc.DEBUG_LEAK)
# Now GC will print info about leaks
```

Q3: When to disable GC

```
# Performance critical code
# GC pauses can cause latency spikes

import gc

gc.disable()  # disable during critical section

# ... performance critical code here ...

gc.collect()  # manually collect
gc.enable()   # re-enable
```

Q4: Detecting cycles

```
import gc

class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

# Create cycle
a = Node(1)
b = Node(2)
a.next = b
b.next = a
```

```
del a, b

# Force collection and check
collected = gc.collect()
print(f"Collected {collected} objects")    # 2
```

6. Cyclic References — Deep Dive

Types of cycles:

```
# Type 1 – Direct cycle (object points to itself)
a = []
a.append(a)
print(a)    # [[...]] ← infinite representation!

# Type 2 – Indirect cycle (A→B→A)
class Node:
    pass
x = Node()
y = Node()
x.ref = y
y.ref = x

# Type 3 – Long chain cycle (A→B→C→A)
a, b, c = Node(), Node(), Node()
a.ref = b
b.ref = c
c.ref = a    # completes the cycle
```

Interview Scenarios:

Q1: Why are cycles a problem?

```
import sys

a = []
a.append(a)    # self referencing list

print(sys.getrefcount(a))    # 3 (a, list element, getrefcount)

del a
```

```
# refcount goes to 2 – NOT 0!  
# Object still references itself  
# Reference counting CANNOT free this  
# GC must step in
```

Q2: How to avoid cycles

```
import weakref  
  
class Parent:  
    def __init__(self):  
        self.child = None  
  
class Child:  
    def __init__(self, parent):  
        # ❌ Strong reference – creates cycle  
        # self.parent = parent  
  
        # ✅ Weak reference – no cycle!  
        self.parent = weakref.ref(parent)  
  
p = Parent()  
c = Child(p)  
p.child = c  
  
# Now when p is deleted, no cycle exists  
del p  
print(c.parent()) # None – parent gone, weakref returns None
```

Q3: `__del__` and cycles — dangerous combo

```
import gc  
  
class Dangerous:  
    def __del__(self):  
        print("cleaning up")  
  
a = Dangerous()  
b = Dangerous()  
a.ref = b  
b.ref = a  
  
del a, b
```

```
# In Python 3.4+: GC handles this
# In Python 3.3-: objects with __del__ in cycle → gc.garbage list
# never freed! 🙄

gc.collect()
print(gc.garbage)  # [] in Python 3.4+ (fixed!)
```

7. Weak References — Deep Dive

Types of weak references:

```
import weakref

class Dog:
    def __init__(self, name):
        self.name = name
    def __repr__(self):
        return f"Dog({self.name})"

d = Dog("Rex")

# 1. Basic weak reference
w = weakref.ref(d)
print(w())          # Dog(Rex) – call it to get object
print(w() is d)     # True

# 2. WeakValueDictionary – values are weak refs
wvd = weakref.WeakValueDictionary()
wvd['rex'] = d
del d
print(dict(wvd))    # {} ← auto cleaned!

# 3. WeakKeyDictionary – keys are weak refs
wkd = weakref.WeakKeyDictionary()
d = Dog("Buddy")
wkd[d] = "some data"
del d
print(dict(wkd))    # {} ← auto cleaned!

# 4. WeakSet
ws = weakref.WeakSet()
d = Dog("Max")
```

```
ws.add(d)
del d
print(list(ws))      # [] ← auto cleaned!
```

Interview Scenarios:

Q1: Build a cache with weak references

```
import weakref

class ExpensiveObject:
    def __init__(self, key):
        self.key = key
        # imagine expensive computation here

    def __repr__(self):
        return f"Expensive({self.key})"

# Cache that doesn't prevent garbage collection
cache = weakref.WeakValueDictionary()

def get_object(key):
    obj = cache.get(key)
    if obj is None:
        print(f"Cache MISS – creating {key}")
        obj = ExpensiveObject(key)
        cache[key] = obj
    else:
        print(f"Cache HIT – returning {key}")
    return obj

a = get_object("x")    # Cache MISS
b = get_object("x")    # Cache HIT

del a, b               # refcount = 0 → object freed
c = get_object("x")    # Cache MISS again (was freed!)
```

Q2: Observer pattern with weak references

```
import weakref

class EventSystem:
    def __init__(self):
```



```

        self._listeners = weakref.WeakSet()

    def subscribe(self, listener):
        self._listeners.add(listener)

    def emit(self, event):
        # dead listeners auto-removed from WeakSet!
        for listener in self._listeners:
            listener(event)

class Button:
    def on_click(self, event):
        print(f"Button heard: {event}")

events = EventSystem()
b = Button()
events.subscribe(b.on_click)

events.emit("click")    # Button heard: click

del b                  # Button deleted
events.emit("click")    # nothing – listener auto removed!

```

Q3: Weak reference callback

```

import weakref

def on_death(ref):
    print(f"Object died! ref={ref}")

class Cat:
    pass

c = Cat()
w = weakref.ref(c, on_death)    # callback when object dies

del c    # → "Object died! ref=<weakref at 0x... dead>"

```

8. Object Lifecycle — Deep Dive

BIRTH	LIFE	DEATH

<code>__new__</code>	<code>__init__</code>	<code>__del__</code>
creates	initializes	cleanup
object	object	object

```
class Lifecycle:
    def __new__(cls):
        print("1. __new__: object created")
        instance = super().__new__(cls)
        return instance

    def __init__(self):
        print("2. __init__: object initialized")

    def __del__(self):
        print("3. __del__: object destroyed")

obj = Lifecycle()
# 1. __new__: object created
# 2. __init__: object initialized

del obj
# 3. __del__: object destroyed
```

Interview Scenarios:

Q1: `__new__` vs `__init__`

```
class Singleton:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            print("Creating instance")
            cls._instance = super().__new__(cls)
        return cls._instance    # always return SAME object

    def __init__(self):
        print("Init called")    # called every time!

a = Singleton()    # Creating instance, Init called
b = Singleton()    # Init called (no "Creating" – reused!)
print(a is b)      # True – same object!
```

Q2: When is `__del__` NOT called?

```
import gc

class Ghost:
    def __del__(self):
        print("Ghost died")

g = Ghost()
g.self_ref = g    # cycle!

del g
# __del__ might not be called immediately
# GC handles it – but timing is unpredictable

gc.collect()      # force it
# "Ghost died" – now called
```

Q3: Object resurrection in `__del__`

```
class Zombie:
    graveyard = []

    def __del__(self):
        print("Dying... but wait!")
        Zombie.graveyard.append(self)    # resurrect!

z = Zombie()
del z    # "Dying... but wait!"

print(len(Zombie.graveyard))    # 1 – it's alive!
# WARNING: Python only allows resurrection ONCE
# Second time __del__ is NOT called
```

9. Interning — Deep Dive

Integer Interning:

```
# Python pre-creates integers -5 to 256
import sys

for i in [-6, -5, 0, 100, 256, 257]:
```

```

a = i
b = i
print(f"{i}: a is b = {a is b}")

# -6: False (below range)
# -5: True (start of range)
# 0: True
# 100: True
# 256: True (end of range)
# 257: False (above range)

```

String Interning Rules:

```

import sys

# Automatically intern
a = "hello"
b = "hello"
print(a is b)    # True (looks like identifier)

# NOT automatically intern
a = "hello world" # has space
b = "hello world"
print(a is b)    # False (maybe – implementation dependent)

a = "hello!"     # has special char
b = "hello!"
print(a is b)    # False

# Force interning
a = sys.intern("hello world")
b = sys.intern("hello world")
print(a is b)    # True ✓

```

Interview Scenarios:

Q1: Integer cache trap

```

# ✗ Dangerous assumption
def compare(a, b):
    return a is b    # WRONG way to compare numbers

print(compare(100, 100))    # True (cached)

```

```
print(compare(1000, 1000)) # False (not cached!)

# ✅ Always use == for value comparison
print(1000 == 1000) # Always True
```

Q2: String interning in loops

```
import sys
import time

words = ["hello"] * 1000000

# Without interning
start = time.time()
result = [w == "hello" for w in words]
print(f"== took: {time.time()-start:.4f}s")

# With interning (faster for repeated comparisons)
interned_words = [sys.intern(w) for w in words]
target = sys.intern("hello")

start = time.time()
result = [w is target for w in interned_words]
print(f"is took: {time.time()-start:.4f}s")
# `is` is faster than `==` for interned strings
```

Q3: Compile time vs runtime interning

```
# Compile time – Python interns automatically
a = "hello"
b = "hello"
print(a is b) # True (compiler optimizes)

# Runtime – built dynamically, NOT interned
a = "hel" + "lo" # compile time concat → interned
b = "hello"
print(a is b) # True (compiler folded it)

x = "hel"
a = x + "lo" # runtime concat → NOT interned
b = "hello"
print(a is b) # False (built at runtime)
```

Q4: Why interning exists — performance

```
# Dictionary lookup uses hash + equality check
# Interned strings skip equality check – just identity!

d = {"name": "Alice", "age": 30}

# Python interns dict keys automatically
# So lookups are faster for string keys
```

10. Mutability — Deep Dive

How Python handles augmented assignment:

```
# Immutable
x = 10
print(id(x))    # 1000
x += 1
print(id(x))    # 2000 ← NEW object!

# Mutable
lst = [1, 2]
print(id(lst))  # 3000
lst += [3]      # calls lst.__iadd__ – modifies IN PLACE
print(id(lst))  # 3000 ← SAME object!
```

Interview Scenarios:

Q1: += behaves differently for mutable vs immutable

```
# Tuple (immutable)
t = (1, 2)
print(id(t))    # 1000
t += (3,)
print(id(t))    # 2000 ← new object

# List (mutable)
l = [1, 2]
print(id(l))    # 3000
```

```
l += [3]
print(id(l))    # 3000 ← same object!
```

Q2: Function argument mutation trap

```
def add_item(lst, item):
    lst.append(item)    # mutates original!
    return lst

original = [1, 2, 3]
result = add_item(original, 4)

print(original)    # [1, 2, 3, 4] ← MUTATED!
print(result is original)    # True – same object!

# Fix – work on a copy
def add_item_safe(lst, item):
    lst = lst.copy()    # work on copy
    lst.append(item)
    return lst

original = [1, 2, 3]
result = add_item_safe(original, 4)
print(original)    # [1, 2, 3] ← unchanged!
```

Q3: Dict and set mutation during iteration

```
d = {'a': 1, 'b': 2, 'c': 3}

# ❌ DANGEROUS
for key in d:
    if key == 'b':
        del d[key]    # RuntimeError: dictionary changed size!

# ✅ Safe – iterate over copy
for key in list(d.keys()):
    if key == 'b':
        del d[key]    # works fine
```

Q4: Frozenset vs Set

```
# Set – mutable, not hashable
s = {1, 2, 3}
```

```

s.add(4)          # ✅
# hash(s)         # ❌ TypeError

# Frozenset – immutable, hashable
fs = frozenset({1, 2, 3})
# fs.add(4)       # ❌ AttributeError
hash(fs)          # ✅ can be used as dict key!

d = {fs: "frozen set key"}  # ✅ works!

```

Q5: Copy vs Deepcopy for nested mutables

```

import copy

original = {"a": [1, 2, 3], "b": [4, 5, 6]}

shallow = original.copy()
deep     = copy.deepcopy(original)

original["a"].append(99)

print(original)  # {'a': [1,2,3,99], 'b': [4,5,6]}
print(shallow)   # {'a': [1,2,3,99], 'b': [4,5,6]} ← affected!
print(deep)      # {'a': [1,2,3], 'b': [4,5,6]} ← safe!

```

Master Cheat Sheet:

Topic	Must Remember
Everything is object	functions, classes, None all have id/type/value
<code>is</code> vs <code>==</code>	identity vs value — always <code>is None</code> never <code>== None</code>
<code>id()</code>	only unique during object's lifetime — can be reused
Refcount	hits 0 → immediately freed — <code>getrefcount</code> adds 1
GC generations	0=new, 1=survived once, 2=long lived
Cycles	refcount can't handle — GC must step in
Weak ref	doesn't increase refcount — object can still die
Interning ints	-5 to 256 cached — never use <code>is</code> for numbers
Interning str	identifier-like strings only — use <code>sys.intern()</code> to force
Mutable default	never use <code>[]</code> or <code>{}</code> as default — use <code>None</code>
<code>+=</code> on mutable	modifies in place — same object
<code>+=</code> on immutable	creates new object — different id

Topic	Must Remember
Shallow copy	copies container only — inner objects shared
Deep copy	copies everything — fully independent

Functions — Deep Internals

1. Functions as Objects

Core Idea:

In Python, functions are **first class objects** — they have identity, type, value just like integers or strings.

```
def greet(name):
    """Says hello"""
    return f"Hello {name}"

# Functions have all object properties
print(type(greet))      # <class 'function'>
print(id(greet))        # memory address
print(greet.__name__)   # 'greet'
print(greet.__doc__)    # 'Says hello'
```

Functions can do everything objects can:

```
# 1. Assign to variable
say_hi = greet
say_hi("Alice")      # Hello Alice

# 2. Store in data structure
funcs = [greet, print, len]
funcs[0]("Bob")      # Hello Bob

# 3. Pass as argument
def run(func, value):
    return func(value)
run(greet, "Charlie") # Hello Charlie
```

```
# 4. Return from function
def get_greeter():
    return greet      # returning function!

fn = get_greeter()
fn("Dave")    # Hello Dave

# 5. Store in dict
actions = {
    'greet': greet,
    'shout': lambda x: x.upper()
}
actions['greet']("Eve")    # Hello Eve
```

Interview Scenarios:

Q1: Prove function is an object

```
def add(a, b):
    return a + b

print(isinstance(add, object))    # True ✓
print(hasattr(add, '__call__'))  # True ✓
print(add.__class__)              # <class 'function'>
```

Q2: Function equality

```
def foo(): pass
def bar(): pass

a = foo
b = foo

print(a is b)    # True ← same function object
print(a is foo)  # True
print(foo is bar)# False ← different function objects
```

Q3: Functions in a dispatch table

```
# Instead of if/elif chain:
def add(a, b): return a + b
```

```

def sub(a, b): return a - b
def mul(a, b): return a * b

# ❌ Bad way
def calculate(op, a, b):
    if op == 'add': return add(a, b)
    elif op == 'sub': return sub(a, b)
    elif op == 'mul': return mul(a, b)

# ✅ Good way – dispatch table
operations = {
    'add': add,
    'sub': sub,
    'mul': mul
}

def calculate(op, a, b):
    return operations[op](a, b)

print(calculate('add', 5, 3))    # 8
print(calculate('mul', 4, 2))    # 8

```

Q4: Dynamic function creation

```

def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

double = make_multiplier(2)
triple = make_multiplier(3)

print(double(5))    # 10
print(triple(5))    # 15
print(type(double)) # <class 'function'>

```

2. Function Attributes

Built-in attributes every function has:

```

def greet(name, greeting="Hello"):
    """Greets a person."""

```

```

    return f"{greeting} {name}"

print(greet.__name__)      # 'greet'
print(greet.__doc__)       # 'Greets a person.'
print(greet.__module__)    # '__main__'
print(greet.__defaults__)  # ('Hello',) ← default args!
print(greet.__code__)      # code object
print(greet.__globals__)   # global namespace dict
print(greet.__dict__)      # custom attributes {}
print(greet.__annotations__)# {} (type hints)
print(greet.__qualname__)  # 'greet'

```

Code object attributes:

```

def add(a, b):
    x = a + b
    return x

code = add.__code__
print(code.co_varnames)    # ('a', 'b', 'x') ← all local vars
print(code.co_argcount)    # 2 ← number of args
print(code.co_filename)    # file where defined
print(code.co_firstlineno) # line number where defined
print(code.co_consts)      # (None,) ← constants used

```

Custom attributes on functions:

```

def counter():
    counter.calls += 1    # store state ON the function!
    return counter.calls

counter.calls = 0        # set custom attribute

print(counter())        # 1
print(counter())        # 2
print(counter())        # 3
print(counter.calls)    # 3

```

Interview Scenarios:

Q1: How to count function calls

```

def track_calls(func):
    def wrapper(*args, **kwargs):
        wrapper.calls += 1
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper

@track_calls
def add(a, b):
    return a + b

add(1, 2)
add(3, 4)
add(5, 6)
print(add.calls)    # 3

```

Q2: Function metadata inspection

```

import inspect

def process(name: str, age: int = 0) -> str:
    """Process a person."""
    return f"{name} is {age}"

# Get all info
print(inspect.signature(process))
# (name: str, age: int = 0) -> str

params = inspect.signature(process).parameters
for name, param in params.items():
    print(f"{name}: default={param.default}, "
          f"annotation={param.annotation}")
# name: default=<empty>, annotation=<class 'str'>
# age: default=0, annotation=<class 'int'>

```

Q3: `__defaults__` vs `__kwdefaults__`

```

def foo(a, b=10, *, c=20):
    pass

print(foo.__defaults__)    # (10,) ← positional defaults
print(foo.__kwdefaults__)  # {'c': 20} ← keyword-only defaults

```

Q4: `__qualname__` for nested functions

```
class MyClass:
    def method(self):
        def inner():
            pass
        print(inner.__qualname__)
        # 'MyClass.method.<locals>.inner'

MyClass().method()
# Shows full qualified path – useful for debugging
```

3. Default Argument Evaluation Time

The Core Rule:

Default arguments are evaluated **ONCE** at function **definition time** — NOT every time the function is called.

```
import time

def log(msg, timestamp=time.time()): # evaluated ONCE!
    print(f"{timestamp}: {msg}")

log("first")    # 1234567890.123: first
time.sleep(2)
log("second")   # 1234567890.123: second ← SAME timestamp!
```

Visualizing when defaults are evaluated:

```
x = 10

def foo(a=x):    # x evaluated HERE at definition
    print(a)

x = 99           # changing x AFTER definition

foo()           # 10 ← uses OLD value of x, not 99!
```

Definition time:

a=x evaluated
x=10 stored

Call time:

function body runs
uses stored default
(ignores current x)

Interview Scenarios:

Q1: The classic mutable default trap

```
# ❌ WRONG
def append_item(item, lst=[]):
    lst.append(item)
    return lst

print(append_item(1))    # [1]
print(append_item(2))    # [1, 2] ← WRONG!
print(append_item(3))    # [1, 2, 3] ← keeps growing!

# WHY: lst=[] created ONCE at definition
# All calls share the SAME list object!

# ✅ CORRECT
def append_item(item, lst=None):
    if lst is None:
        lst = []    # fresh list every call
    lst.append(item)
    return lst

print(append_item(1))    # [1]
print(append_item(2))    # [2] ✅
```

Q2: Default dict trap

```
# ❌ WRONG
def add_key(key, value, d={}):
    d[key] = value
    return d

print(add_key('a', 1))    # {'a': 1}
print(add_key('b', 2))    # {'a': 1, 'b': 2} ← accumulated!
```

```
#  CORRECT
def add_key(key, value, d=None):
    if d is None:
        d = {}
    d[key] = value
    return d
```

Q3: When mutable default is intentional

```
# Sometimes you WANT shared state
def make_cache():
    cache = {}
    def cached_func(key, value=None, _cache=cache):
        # _cache intentionally shared!
        if value is not None:
            _cache[key] = value
        return _cache.get(key)
    return cached_func

cache_fn = make_cache()
cache_fn('x', 10)
print(cache_fn('x'))    # 10 ← remembered!
```

Q4: Timestamp default trap

```
from datetime import datetime

#  WRONG – timestamp frozen at definition
def log_event(event, time=datetime.now()):
    print(f"{time}: {event}")

#  CORRECT – fresh timestamp each call
def log_event(event, time=None):
    if time is None:
        time = datetime.now()
    print(f"{time}: {event}")
```

4. Closures & Free Variables

What is a Closure?

A closure is a function that **remembers variables** from its enclosing scope even after that scope has finished.

```
def outer():
    x = 10          # x is a FREE VARIABLE

    def inner():
        print(x)    # inner REMEMBERS x

    return inner    # outer finished, but x lives on!

fn = outer()       # outer() done – x should be gone?
fn()               # 10 ← x is still alive! (closure)
```

Free variables — how to inspect:

```
def make_adder(n):
    def adder(x):
        return x + n    # n is free variable
    return adder

add5 = make_adder(5)

print(add5.__code__.co_freevars)  # ('n',) ← free variables
print(add5.__closure__)           # (<cell at 0x...>,)
print(add5.__closure__[0].cell_contents)  # 5 ← actual value!
```

Interview Scenarios:

Q1: Classic closure loop trap

```
# ❌ WRONG
funcs = []
for i in range(5):
    funcs.append(lambda: i)    # i is free variable!

print([f() for f in funcs])    # [4, 4, 4, 4, 4] ← ALL 4!
# WHY: all lambdas share SAME i
# by the time they run, loop finished, i=4

# ✅ Fix 1 – default argument captures value
funcs = []
```

```

for i in range(5):
    funcs.append(lambda i=i: i)    # i captured at creation!

print([f() for f in funcs])    # [0, 1, 2, 3, 4] ✓

# ✓ Fix 2 – closure factory
def make_func(i):
    def fn():
        return i
    return fn

funcs = [make_func(i) for i in range(5)]
print([f() for f in funcs])    # [0, 1, 2, 3, 4] ✓

```

Q2: Closure with mutable state

```

def make_counter():
    count = 0

    def increment():
        nonlocal count    # ← must declare nonlocal to modify!
        count += 1
        return count

    def reset():
        nonlocal count
        count = 0

    return increment, reset

inc, rst = make_counter()
print(inc())    # 1
print(inc())    # 2
print(inc())    # 3
rst()
print(inc())    # 1 ← reset worked!

```

Q3: `nonlocal` vs `global`

```

x = "global"

def outer():
    x = "outer"

```

```

def inner():
    nonlocal x      # refers to outer's x
    x = "inner"
    print(x)        # inner

inner()
print(x)            # inner ← outer's x was modified!

outer()
print(x)            # global ← global x unchanged

```

Q4: Closure cell sharing

```

def outer():
    x = []          # mutable free variable

    def add(item):
        x.append(item)  # no nonlocal needed – mutating, not rebinding

    def show():
        return x

    return add, show

add, show = outer()
add(1)
add(2)
add(3)
print(show())      # [1, 2, 3] ← shared state via closure!

```

Q5: Multiple closures sharing same cell

```

def make_pair():
    value = [0]     # use list to allow mutation

    def get():
        return value[0]

    def set(new_val):
        value[0] = new_val

    return get, set

get, set = make_pair()

```

```
print(get())    # 0
set(42)
print(get())    # 42 ← shared!
```

5. Decorators

What is a Decorator?

A decorator is a function that **takes a function and returns a new function** — wrapping extra behavior around it.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before")
        result = func(*args, **kwargs)
        print("After")
        return result
    return wrapper

# Manual way
def greet(name):
    print(f"Hello {name}")

greet = my_decorator(greet)  # wrapping manually

# @ syntax – exact same thing!
@my_decorator
def greet(name):
    print(f"Hello {name}")
```

Interview Scenarios:

Q1: Timer decorator

```
import time
from functools import wraps

def timer(func):
    @wraps(func)    # preserves original function metadata
    def wrapper(*args, **kwargs):
        start = time.time()
```

```

        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end-start:.4f}s")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(1)

slow_function()    # slow_function took 1.0012s

```

Q2: Decorator with arguments

```

from functools import wraps

def repeat(times):
    # outer – accepts args
    def decorator(func):
        # middle – accepts function
        @wraps(func)
        def wrapper(*args, **kwargs):
            # inner – runs each call
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator

@repeat(times=3)
def greet(name):
    print(f"Hello {name}")

greet("Alice")
# Hello Alice
# Hello Alice
# Hello Alice

```

Q3: Class-based decorator

```

from functools import wraps

class CountCalls:
    def __init__(self, func):
        wraps(func)(self)    # copy metadata
        self.func = func
        self.calls = 0

```

```

def __call__(self, *args, **kwargs):
    self.calls += 1
    print(f"Call #{self.calls}")
    return self.func(*args, **kwargs)

@CountCalls
def add(a, b):
    return a + b

add(1, 2)    # Call #1
add(3, 4)    # Call #2
print(add.calls)    # 2

```

Q4: Stacking decorators

```

from functools import wraps

def bold(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return "<b>" + func(*args, **kwargs) + "</b>"
    return wrapper

def italic(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return "<i>" + func(*args, **kwargs) + "</i>"
    return wrapper

@bold
@italic
def greet(name):
    return f"Hello {name}"

print(greet("Alice"))    # <b><i>Hello Alice</i></b>

# Order matters – applied bottom up:
# italic applied first → then bold wraps italic

```

Q5: Decorator to validate input

```

from functools import wraps

```

```

def validate_positive(*arg_names):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            import inspect
            bound = inspect.signature(func).bind(*args, **kwargs)
            for name in arg_names:
                if name in bound.arguments:
                    if bound.arguments[name] < 0:
                        raise ValueError(f"{name} must be positive!")
            return func(*args, **kwargs)
        return wrapper
    return decorator

@validate_positive('a', 'b')
def add(a, b):
    return a + b

print(add(5, 3))    # 8 ✓
print(add(-1, 3))   # ValueError: a must be positive! ✗

```

Q6: `@wraps` — why it matters

```

from functools import wraps

# ✗ Without @wraps
def decorator(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@decorator
def greet():
    """Says hello"""
    pass

print(greet.__name__)    # 'wrapper' ← WRONG!
print(greet.__doc__)     # None ← LOST!

# ✓ With @wraps
def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)

```

```

    return wrapper

@decorator
def greet():
    """Says hello"""
    pass

print(greet.__name__)    # 'greet' ✓
print(greet.__doc__)    # 'Says hello' ✓

```

6. **functools** — Deep Dive

functools.wraps :

Copies function metadata from original to wrapper.
(Already covered above in Q6)

functools.partial :

Creates a new function with some arguments **pre-filled**.

```

from functools import partial

def power(base, exponent):
    return base ** exponent

# Pre-fill exponent=2
square = partial(power, exponent=2)
cube   = partial(power, exponent=3)

print(square(5))    # 25
print(cube(3))      # 27

```

functools.lru_cache :

Caches function results — same inputs return **cached output** without recalculating.

```

from functools import lru_cache

@lru_cache(maxsize=128)

```



```
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(100))    # instant! (cached results)
print(fibonacci.cache_info())
# CacheInfo(hits=98, misses=101, maxsize=128, cursize=101)
```

Interview Scenarios:

Q1: **partial** for event handlers

```
from functools import partial

def handle_event(event_type, data, source):
    print(f"{event_type} from {source}: {data}")

# Pre-fill event_type and source
handle_click = partial(handle_event, "click", source="button")
handle_hover = partial(handle_event, "hover", source="menu")

handle_click("submit form")    # click from button: submit form
handle_hover("dropdown open") # hover from menu: dropdown open
```

Q2: **lru_cache** performance comparison

```
from functools import lru_cache
import time

# ❌ Without cache – exponential time!
def fib_slow(n):
    if n < 2: return n
    return fib_slow(n-1) + fib_slow(n-2)

# ✅ With cache – linear time!
@lru_cache(maxsize=None)
def fib_fast(n):
    if n < 2: return n
    return fib_fast(n-1) + fib_fast(n-2)

start = time.time()
fib_slow(35)
```

```
print(f"Slow: {time.time()-start:.3f}s")    # ~3 seconds!

start = time.time()
fib_fast(35)
print(f"Fast: {time.time()-start:.6f}s")    # 0.000001s!
```

Q3: `lru_cache` limitations

```
from functools import lru_cache

@lru_cache(maxsize=128)
def process(data):
    return sum(data)

# ❌ Unhashable types cannot be cached
process([1, 2, 3])    # TypeError: unhashable type: 'list'

# ✅ Only hashable types work
process((1, 2, 3))    # 6 ← tuple works!
process(frozenset({1,2})) # works!
```

Q4: `functools.reduce`

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

# Multiply all numbers
result = reduce(lambda x, y: x * y, numbers)
print(result)    # 120 (1*2*3*4*5)

# Running total
result = reduce(lambda acc, x: acc + x, numbers, 0)
print(result)    # 15
```

Q5: Cache with timeout (custom)

```
from functools import wraps
import time

def timed_cache(seconds):
    def decorator(func):
        cache = {}
```

```

@wraps(func)
def wrapper(*args):
    now = time.time()
    if args in cache:
        result, timestamp = cache[args]
        if now - timestamp < seconds:
            print("Cache HIT")
            return result
        print("Cache MISS")
        result = func(*args)
        cache[args] = (result, now)
        return result
    return wrapper
return decorator

@timed_cache(seconds=5)
def get_data(key):
    return f"data for {key}"

get_data("x")    # Cache MISS
get_data("x")    # Cache HIT
time.sleep(6)
get_data("x")    # Cache MISS (expired!)

```

7. Lambda Internals

What is Lambda?

A lambda is an **anonymous, single-expression function**.

```

# Regular function
def add(a, b):
    return a + b

# Exact same as lambda
add = lambda a, b: a + b

```

Lambda internals:

```
fn = lambda x: x * 2
```

```
print(type(fn))          # <class 'function'>
print(fn.__name__)       # '<lambda>' ← anonymous
print(fn.__code__.co_varnames) # ('x',)
```

Interview Scenarios:

Q1: Lambda in sorting

```
people = [
    {'name': 'Alice', 'age': 30},
    {'name': 'Bob', 'age': 25},
    {'name': 'Charlie', 'age': 35}
]

# Sort by age
people.sort(key=lambda p: p['age'])
print([p['name'] for p in people])
# ['Bob', 'Alice', 'Charlie']

# Sort by name length then alphabetically
people.sort(key=lambda p: (len(p['name']), p['name']))
```

Q2: Lambda limitations

```
# ❌ Cannot have statements
fn = lambda x: if x > 0: x else -x    # SyntaxError!

# ✅ Can use ternary
fn = lambda x: x if x > 0 else -x    # works!

# ❌ Cannot have multiple lines
fn = lambda x: (
    y = x * 2    # SyntaxError!
    return y
)

# ✅ Use regular function instead
def fn(x):
    y = x * 2
    return y
```

Q3: Lambda vs function — when to use which

```
# ✓ Lambda – short, throwaway, used once
numbers = [1, -2, 3, -4, 5]
positive = filter(lambda x: x > 0, numbers)

# ✓ Regular function – complex, reused, needs docstring
def is_valid_email(email):
    """Validates email format."""
    import re
    return bool(re.match(r'^[\w.-]+@[\w.-]+\.\w+$', email))
```

Q4: Immediately invoked lambda

```
result = (lambda x, y: x + y)(5, 3)
print(result)    # 8

# Useful for complex inline initialization
matrix = (lambda n: [[0]*n for _ in range(n)])(3)
print(matrix)    # [[0,0,0],[0,0,0],[0,0,0]]
```

8. `__call__`

What is `__call__`?

| Makes any object **callable** — usable like a function with `()`.

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

double = Multiplier(2)
triple = Multiplier(3)

print(double(5))    # 10 ← calling object like function!
print(triple(5))    # 15

print(callable(double))    # True
```

Interview Scenarios:

Q1: Stateful callable

```
class Counter:
    def __init__(self):
        self.count = 0

    def __call__(self):
        self.count += 1
        return self.count

count = Counter()
print(count())    # 1
print(count())    # 2
print(count())    # 3
print(count.count) # 3 ← state preserved!
```

Q2: `__call__` as decorator

```
class Retry:
    def __init__(self, times):
        self.times = times

    def __call__(self, func):
        from functools import wraps
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(self.times):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    print(f"Attempt {attempt+1} failed: {e}")
            raise Exception("All attempts failed!")
        return wrapper

@Retry(times=3)
def unstable_function():
    import random
    if random.random() < 0.7:
        raise ValueError("Random failure!")
    return "Success!"
```

Q3: Checking if object is callable

```
def foo(): pass
class Bar:
    def __call__(self): pass
class Baz:
    pass

print(callable(foo))      # True ← function
print(callable(Bar()))    # True ← has __call__
print(callable(Baz()))    # False ← no __call__
print(callable(42))       # False ← int not callable
print(callable(int))      # True ← class itself callable!
```

Q4: Pipeline using `__call__`

```
class Pipeline:
    def __init__(self):
        self.steps = []

    def add(self, func):
        self.steps.append(func)
        return self  # allows chaining!

    def __call__(self, data):
        for step in self.steps:
            data = step(data)
        return data

pipe = Pipeline()
pipe.add(lambda x: x.strip()) \
    .add(lambda x: x.lower()) \
    .add(lambda x: x.replace(" ", "_"))

print(pipe(" Hello World "))  # hello_world
```

9. Stack Frames

What is a Stack Frame?

Every function call creates a **frame** — a snapshot of that function's local state.

```

import inspect

def inner():
    frame = inspect.currentframe()
    print(f"Function: {frame.f_code.co_name}")
    print(f"Line: {frame.f_lineno}")
    print(f"Locals: {frame.f_locals}")

def outer():
    x = 10
    inner()

outer()
# Function: inner
# Line: 3
# Locals: {'frame': <frame object>}

```

Call stack visualization:

```

def c():
    pass      # Frame C ← top of stack

def b():
    c()       # Frame B

def a():
    b()       # Frame A

a()          # Frame main ← bottom of stack

# Stack at deepest point:
#
# | Frame C | ← currently executing
# | Frame B |
# | Frame A |
# |  main  | ← bottom
#

```

Interview Scenarios:

Q1: Inspecting call stack


```

import traceback
import inspect

def deep_function():
    # Print full call stack
    traceback.print_stack()

    # Or inspect frame by frame
    frame = inspect.currentframe()
    while frame:
        print(f"→ {frame.f_code.co_name} "
              f"(line {frame.f_lineno})")
        frame = frame.f_back    # go to caller's frame

def middle():
    deep_function()

def start():
    middle()

start()

```

Q2: Frame locals and globals

```

import inspect

def foo():
    x = 10
    y = 20
    frame = inspect.currentframe()
    print(frame.f_locals)    # {'x': 10, 'y': 20, 'frame': ...}
    print('foo' in frame.f_globals) # True

foo()

```

Q3: Memory — frames and references

```

import inspect

def dangerous():
    frame = inspect.currentframe()
    # ⚠ NEVER store frames long-term!
    # Frame holds references to all local variables

```

```

# Storing frame = memory leak (prevents GC)
return frame # ❌ dangerous!

# ✅ Always delete frame references
def safe():
    frame = inspect.currentframe()
    try:
        info = frame.f_lineno # get what you need
    finally:
        del frame # always clean up!

```

10. Recursion & Recursion Limits

What is Recursion?

| A function that **calls itself** to solve smaller versions of the same problem.

```

def factorial(n):
    if n <= 1: # BASE CASE – stops recursion
        return 1
    return n * factorial(n-1) # RECURSIVE CASE

print(factorial(5)) # 120

```

Call stack during recursion:

```

factorial(5)
└ factorial(4)
  └ factorial(3)
    └ factorial(2)
      └ factorial(1)
        └ returns 1
          returns 2
            returns 6
              returns 24
                returns 120

```

Recursion limit:

```

import sys

print(sys.getrecursionlimit())    # 1000 (default)

# Change it
sys.setrecursionlimit(5000)

# Hit the limit
def infinite():
    return infinite()

infinite()    # RecursionError: maximum recursion depth exceeded

```

Interview Scenarios:

Q1: Recursion vs Iteration

```

# Recursive – elegant but stack risk
def fib_recursive(n):
    if n < 2: return n
    return fib_recursive(n-1) + fib_recursive(n-2)

# Iterative – safe, faster
def fib_iterative(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a

# With memoization – best of both
from functools import lru_cache

@lru_cache(maxsize=None)
def fib_memo(n):
    if n < 2: return n
    return fib_memo(n-1) + fib_memo(n-2)

```

Q2: Tail recursion (Python doesn't optimize it!)

```

# Python does NOT do tail call optimization
# This will still hit recursion limit!

```

```
def tail_factorial(n, acc=1):
    if n <= 1: return acc
    return tail_factorial(n-1, n * acc)    # tail call

# ❌ Still hits limit at ~1000
tail_factorial(5000)    # RecursionError!

# ✅ Convert to loop instead
def loop_factorial(n):
    acc = 1
    while n > 1:
        acc *= n
        n -= 1
    return acc
```

Q3: Flatten nested list with recursion

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))    # recurse!
        else:
            result.append(item)
    return result

nested = [1, [2, [3, [4, [5]]]], 6]
print(flatten(nested))    # [1, 2, 3, 4, 5, 6]

# Iterative version (safer for deep nesting)
def flatten_safe(lst):
    stack = [lst]
    result = []
    while stack:
        current = stack.pop()
        if isinstance(current, list):
            stack.extend(reversed(current))
        else:
            result.append(current)
    return result
```

Q4: Mutual recursion

```
def is_even(n):
    if n == 0: return True
    return is_odd(n - 1)

def is_odd(n):
    if n == 0: return False
    return is_even(n - 1)

print(is_even(4))    # True
print(is_odd(3))     # True
# △ Hits recursion limit for large n!
# Better: n % 2 == 0
```

Q5: Recursive data structures

```
# Tree traversal
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def inorder(node):
    if node is None:
        return []
    return inorder(node.left) + [node.val] + inorder(node.right)

root = TreeNode(4,
    TreeNode(2, TreeNode(1), TreeNode(3)),
    TreeNode(6, TreeNode(5), TreeNode(7))
)

print(inorder(root))    # [1, 2, 3, 4, 5, 6, 7]
```

Master Cheat Sheet:

Topic	Must Remember
Functions as objects	assignable, passable, storable
<code>__defaults__</code>	evaluated ONCE at definition time
Mutable default	always use <code>None</code> as default, create inside
Closures	remember enclosing scope via cells

Topic	Must Remember
<code>nonlocal</code>	needed to REBIND free variable
Loop closure trap	use default arg or factory function
<code>@wraps</code>	always use — preserves metadata
Decorator order	applied bottom-up, executes top-down
<code>partial</code>	pre-fill arguments
<code>lru_cache</code>	hashable args only, use <code>maxsize=None</code> for unlimited
Lambda	anonymous, single expression, no statements
<code>__call__</code>	makes object callable like function
Stack frame	each call creates frame — holds locals
Recursion limit	default 1000 — use iteration for deep recursion
Tail recursion	Python does NOT optimize it

In Python, **attribute lookup** follows a strict resolution algorithm when you access something like:

```
obj.attr
```

Python must determine **where** `attr` **exists**. This process is governed by the **Attribute Lookup Order** and the **Method Resolution Order (MRO)**.

I'll first show the **full lookup flow**, then explain each step in detail.

1 Full Attribute Lookup Flow

When you access:

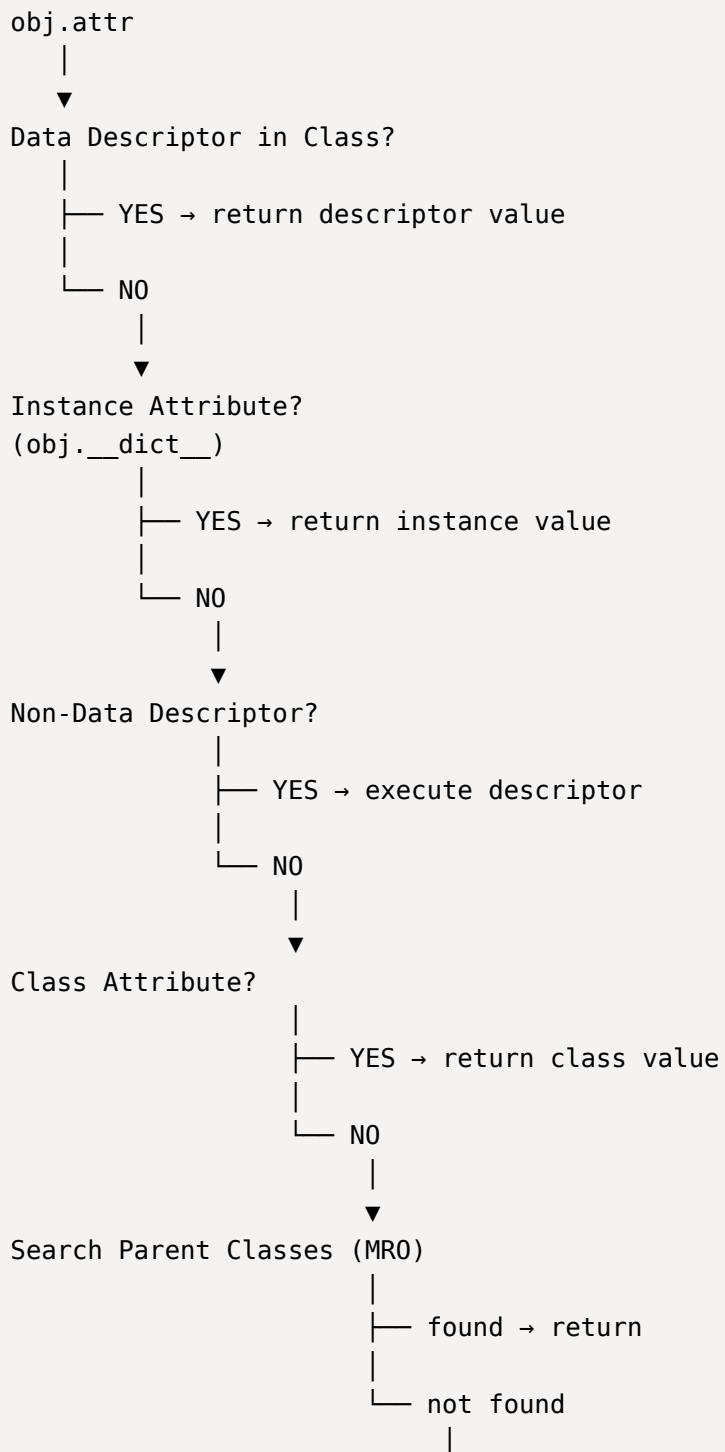
```
obj.attr
```

Python searches in this order:

1. Data Descriptor in the Class
2. Instance Dictionary (`obj.__dict__`)
3. Non-Data Descriptor in the Class

4. Class Dictionary (Class.__dict__)
5. Parent Classes via MRO
6. __getattr__() fallback

Visual flow:



Call `__getattr__()`

If still not found:

`AttributeError`

2 Step 1 — Data Descriptors

Python **first checks data descriptors defined in the class**.

A **data descriptor** is any object that implements:

`__get__()` AND (`__set__()` or `__delete__()`)

Example: `@property`

Example:

```
class A:
    @property
    def x(self):
        return 100

obj = A()
print(obj.x)
```

Even if instance has `x`, **descriptor wins**.

Why?

Because data descriptors have **highest priority**.

Visual:

```
Class
└─ x → property descriptor

Instance
└─ x → value

Result → property wins
```

3 Step 2 — Instance Dictionary

If no data descriptor exists, Python checks:


```
obj.__dict__
```

Example:

```
class A:
    pass

obj = A()
obj.x = 10

print(obj.x)
```

Memory:

```
obj.__dict__ = {
    "x": 10
}
```

Python returns:

```
10
```

4 Step 3 — Non-Data Descriptors

A **non-data descriptor** implements:

```
__get__()
```

but **not** `__set__()`.

Example: **methods**

```
class A:
    def hello(self):
        print("hi")
```

Internally:

```
A.__dict__["hello"] → function descriptor
```

When accessed:

```
obj.hello
```

Python converts it to:

```
bound method
```

Visualization:

```
function descriptor
  |
  ▼
bind obj → method
```

5 Step 4 — Class Dictionary

If attribute isn't in the instance, Python checks:

```
Class.__dict__
```

Example:

```
class A:
    x = 50

obj = A()
print(obj.x)
```

Search:

```
obj.__dict__ → not found
A.__dict__ → found
```

Result:

```
50
```

6 Step 5 — Parent Classes (MRO)

If not in class, Python searches **parent classes using MRO**.

Example:

```
class A:
    x = 10
```

```
class B(A):  
    pass  
  
obj = B()  
print(obj.x)
```

Search order:

```
B.__dict__  
A.__dict__  
object.__dict__
```

This order is defined by **MRO**.

You can see it using:

```
print(B.__mro__)
```

Output:

```
(B, A, object)
```

7 Step 6 — `__getattr__` Fallback

If attribute still isn't found, Python calls:

```
__getattr__(self, name)
```

Example:

```
class A:  
    def __getattr__(self, name):  
        return "Not Found"  
  
obj = A()  
print(obj.xyz)
```

Output:

```
Not Found
```

Why?

Because `xyz` was not found anywhere else.

8 Important Special Case — `__getattr__`

Before all lookup happens, Python actually calls:

```
__getattr__()
```

Example:

```
class A:
    def __getattr__(self, name):
        print("Accessing:", name)
        return super().__getattr__(name)
```

This method intercepts **all attribute access**.

Flow becomes:

```
obj.attr
|
▼
__getattr__()
|
▼
normal lookup
```

9 Complete Real Lookup Order

The **true complete lookup pipeline**:

```
obj.attr
|
▼
__getattr__()
|
▼
Data Descriptor
|
▼
Instance Dictionary
|
▼
Non-Data Descriptor
|
▼
```

```
Class Dictionary
|
▼
Parent Classes (MRO)
|
▼
__getattr__()
|
▼
AttributeError
```

10 Real Python Example Demonstrating Order

```
class Descriptor:
    def __get__(self, obj, owner):
        print("descriptor called")
        return 100

class A:
    x = Descriptor()

obj = A()
obj.x = 10

print(obj.x)
```

Output:

```
descriptor called
100
```

Why?

Because **data descriptors override instance attributes**.

1 1 Interview One-Line Answer

Python attribute lookup follows this order: `__getattribute__` → data descriptors → instance dictionary → non-data descriptors → class dictionary → parent classes via MRO → `__getattr__`.

If you'd like, I can also show you a **visual memory diagram of descriptors vs instance attributes**, which makes this concept **extremely easy to remember for interviews**.

Iterators and Generators in Python

1. The Iteration Protocol (Foundation)

Python's iteration system is built on two core methods:

- `__iter__()` — returns the iterator object itself
- `__next__()` — returns the next value, raises `StopIteration` when done

Any object implementing both is an **iterator**. Any object implementing `__iter__` (returning an iterator) is an **iterable**.

```
# What happens internally when you write: for x in [1, 2, 3]
my_list = [1, 2, 3]
iterator = iter(my_list)      # calls my_list.__iter__()
print(next(iterator))         # calls iterator.__next__() → 1
print(next(iterator))         # → 2
print(next(iterator))         # → 3
# next(iterator)              # → raises StopIteration
```

2. Building a Custom Iterator (from scratch)

```
class CountUp:
    def __init__(self, start, stop):
        self.current = start
        self.stop = stop

    def __iter__(self):
        return self          # the object IS its own iterator

    def __next__(self):
        if self.current >= self.stop:
            raise StopIteration
        value = self.current
```

```

        self.current += 1
        return value

for n in CountUp(1, 5):
    print(n)    # 1, 2, 3, 4

```

Internally, `for n in CountUp(1, 5)` is sugar for:

```

_iter = CountUp(1, 5).__iter__()
while True:
    try:
        n = _iter.__next__()
    except StopIteration:
        break

```

3. Iterable vs Iterator — the key distinction

	Iterable	Iterator
Has <code>__iter__</code>	✓	✓
Has <code>__next__</code>	✗	✓
Can loop multiple times	✓	✗ (exhausted after one pass)
Examples	<code>list</code> , <code>str</code> , <code>dict</code>	<code>map</code> , <code>zip</code> , <code>file objects</code>

```

my_list = [1, 2, 3]          # Iterable (not an iterator)
it = iter(my_list)           # Iterator (wraps the list)

print(iter(my_list) is my_list) # False – list creates a NEW iterator each time
print(iter(it) is it)          # True  – iterators return themselves

```

4. Generators — Iterators Made Easy

A **generator function** uses `yield` instead of `return`. Calling it returns a **generator object** (which is both an iterable and an iterator).

```

def count_up(start, stop):
    while start < stop:
        yield start          # pauses here, sends value out
        start += 1           # resumes here on next()

gen = count_up(1, 5)

```

```
print(next(gen))    # 1
print(next(gen))    # 2
```

What happens internally at `yield` ?

When `yield` is hit, Python:

1. **Freezes** the function's entire stack frame (local variables, instruction pointer)
2. **Sends** the yielded value to the caller
3. **Suspends** execution — the function is paused, not terminated
4. On the next `next()` call, execution **resumes** from exactly where it left off

This is fundamentally different from `return`, which destroys the stack frame.

5. Generator Expressions

Like list comprehensions, but **lazy** — values are computed on demand:

```
# List comprehension – builds entire list in memory
squares_list = [x**2 for x in range(1_000_000)]    # ~8MB

# Generator expression – computes one value at a time
squares_gen = (x**2 for x in range(1_000_000))    # ~200 bytes

print(next(squares_gen))    # 0
print(next(squares_gen))    # 1
```

Use generators when you **don't need all values at once** — they're essential for large or infinite sequences.

6. `yield from` — Delegating to Sub-Generators

`yield from` lets a generator delegate to another iterable, forwarding values transparently:

```
def inner():
    yield 1
    yield 2

def outer():
    yield 'a'
    yield from inner()    # delegates to inner generator
    yield 'b'

list(outer())    # ['a', 1, 2, 'b']
```


`yield from` also **tunnels** `.send()` and `.throw()` calls through to the inner generator (more on that below).

7. Advanced: Two-Way Communication with `.send()`

Generators aren't just output pipelines — you can **send values back in**:

```
def accumulator():
    total = 0
    while True:
        value = yield total    # yields total OUT, receives value IN
        if value is None:
            break
        total += value

gen = accumulator()
next(gen)           # must prime the generator (advance to first yield)
gen.send(10)        # total = 10, yields 10
gen.send(20)        # total = 30, yields 30
gen.send(5)         # total = 35, yields 35
```

The `yield` expression has two roles simultaneously:

- Right side: receives the sent value (`value = yield ...`)
- Left side: emits a value outward (`yield total`)

Priming rule: You must call `next(gen)` (or `gen.send(None)`) once before sending values — this runs the function up to the first `yield`.

8. Advanced: `.throw()` and `.close()`

```
def safe_gen():
    try:
        while True:
            yield
    except ValueError as e:
        print(f"Caught: {e}")
        yield "recovered"
    finally:
        print("Generator cleaned up")    # always runs on close

gen = safe_gen()
next(gen)
gen.throw(ValueError, "something broke") # injects exception at yield point
gen.close()    # raises GeneratorExit inside the generator → triggers finally
```

- `.throw(exc)` — raises an exception at the point where the generator is paused
- `.close()` — throws `GeneratorExit`, allowing cleanup via `finally`

9. Generator State Machine (Internal States)

A generator object can be in one of four states (inspectable via `inspect.getgeneratorstate()`):

```
GEN_CREATED    → just created, never run
    ↓ next()
GEN_RUNNING    → currently executing
    ↓ yield
GEN_SUSPENDED  → paused at a yield
    ↓ StopIteration / return
GEN_CLOSED     → exhausted or closed
```

```
import inspect

def my_gen():
    yield 1
    yield 2

g = my_gen()
print(inspect.getgeneratorstate(g))  # GEN_CREATED
next(g)
print(inspect.getgeneratorstate(g))  # GEN_SUSPENDED
next(g)
try: next(g)
except StopIteration: pass
print(inspect.getgeneratorstate(g))  # GEN_CLOSED
```

10. `return` inside a Generator

A `return` in a generator doesn't return a value to the caller directly — it raises `StopIteration` with the value as the exception's `.value` attribute:

```
def gen_with_return():
    yield 1
    yield 2
    return "final value"  # becomes StopIteration.value

g = gen_with_return()
next(g)  # 1
next(g)  # 2
```

```

try:
    next(g)
except StopIteration as e:
    print(e.value)    # "final value"

```

This is how `yield from` captures return values from sub-generators.

11. Practical Patterns

Infinite sequences (impossible with lists)

```

def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

fib = fibonacci()
[next(fib) for _ in range(8)]    # [0, 1, 1, 2, 3, 5, 8, 13]

```

Memory-efficient data pipelines

```

def read_large_file(path):
    with open(path) as f:
        for line in f:            # file objects are iterators
            yield line.strip()

def filter_lines(lines, keyword):
    for line in lines:
        if keyword in line:
            yield line

def pipeline(path, keyword):
    lines = read_large_file(path)
    return filter_lines(lines, keyword)

# Only one line lives in memory at a time – no matter the file size

```

Coroutine-style generators (pre-async/await)

```

def logger(name):
    print(f"[{name}] started")
    while True:
        msg = yield

```

```

        print(f"[{name}] {msg}")

log = logger("APP")
next(log)          # prime
log.send("Hello")  # [APP] Hello
log.send("World")  # [APP] World

```

Summary Map

```

Iterable → has __iter__()
    |
    └─> Iterator → has __iter__() + __next__()
            |
            └─> Generator → created by a function with yield
                            |
                            └─> supports .send(), .throw(), .close()
                                    |
                                    └─> basis for async/await in Python

```

3.5+

Python Classes & OOP Internals — Complete Interview Notes

1. Class Creation Process

What happens when Python sees a `class` statement?

```

class MyClass:
    x = 10
    def hello(self): pass

```

Python executes this in **three steps** internally:

Step 1 — Determine the metaclass

```

# Python looks for metaclass in this order:
# 1. Explicit `metaclass=` keyword in class definition
# 2. Metaclass of the first base class
# 3. Defaults to `type`

```

```
class MyClass(metaclass=type):    # this is what Python does by default
    pass
```

Step 2 — Prepare the namespace

```
# Python calls metaclass.__prepare__() to get the class namespace dict
namespace = type.__prepare__('MyClass', (object,), {})
# returns an empty dict {} by default
```

Step 3 — Execute the class body, then call metaclass()

```
# class body runs inside the namespace dict
exec(class_body, globals(), namespace)
# namespace is now {'x': 10, 'hello': <function>}

# Finally, type() is called to BUILD the class object
MyClass = type('MyClass', (object,), namespace)
```

type() — The class factory

```
# type() with 3 args dynamically creates a class
Dog = type('Dog', (object,), {
    'sound': 'woof',
    'speak': lambda self: print(self.sound)
})

d = Dog()
d.speak()    # woof

# This is IDENTICAL to:
class Dog:
    sound = 'woof'
    def speak(self): print(self.sound)
```

What IS a class object?

```
class Person:
    species = "human"

# Person is itself an OBJECT – an instance of `type`
print(type(Person))    # <class 'type'>
print(isinstance(Person, type))    # True

# Every class is an instance of its metaclass
```

```
# type is its own metaclass
print(type(type))          # <class 'type'>
```

Metaclass — the class of a class

```
class Meta(type):
    def __new__(mcs, name, bases, namespace):
        print(f"Creating class: {name}")
        cls = super().__new__(mcs, name, bases, namespace)
        return cls

class MyClass(metaclass=Meta):    # "Creating class: MyClass"
    pass

# Metaclass hooks (in order of execution):
# Meta.__prepare__ → prepare namespace
# Meta.__new__     → create the class object
# Meta.__init__    → initialize the class object
# Meta.__call__    → called when class is INSTANTIATED (MyClass())
```

2. Class vs Instance Attributes

The fundamental rule: Two separate namespaces

```
class Dog:
    species = "Canis lupus"    # class attribute – lives in Dog.__dict__

    def __init__(self, name):
        self.name = name      # instance attribute – lives in instance.__dict__

d1 = Dog("Rex")
d2 = Dog("Buddy")

print(Dog.__dict__)          # {'species': 'Canis lupus', '__init__': ...}
print(d1.__dict__)           # {'name': 'Rex'}
print(d2.__dict__)           # {'name': 'Buddy'}
```

Attribute Lookup Order (LEGB for attributes)

```
# Python looks up attributes in this order:
# 1. Data descriptors from the class (e.g., property with __set__)
```

```
# 2. Instance __dict__
# 3. Non-data descriptors and other class attributes

dl.name          # found in dl.__dict__          → 'Rex'
dl.species        # NOT in dl.__dict__, found in Dog.__dict__ → 'Canis lupus'
```

The shadow/override trap

```
class Counter:
    count = 0                # shared class attribute

    def increment(self):
        self.count += 1      # ⚠ THIS IS A TRAP

c1 = Counter()
c2 = Counter()

c1.increment()
print(c1.count)    # 1 - instance attribute created on c1
print(c2.count)    # 0 - still reading class attribute
print(Counter.count) # 0 - class attribute UNCHANGED

# self.count += 1 is really:
# self.count = self.count + 1
# reads class attr (0), creates NEW instance attr (1)
# to actually share: Counter.count += 1
```

Mutable class attributes — the classic gotcha

```
class Team:
    members = []             # SHARED mutable – one list for ALL instances

t1 = Team()
t2 = Team()
t1.members.append("Alice")

print(t2.members)    # ['Alice'] ← t2 is affected!

# Fix: create per-instance list in __init__
class Team:
    def __init__(self):
        self.members = []    # each instance gets its OWN list
```

Class attribute use-cases

```
class Circle:
    pi = 3.14159          # shared constant – sensible class attribute
    _instance_count = 0   # shared counter

    def __init__(self, r):
        self.radius = r
        Circle._instance_count += 1    # explicit class-level mutation

    @classmethod
    def get_count(cls):
        return cls._instance_count

c1 = Circle(5)
c2 = Circle(3)
print(Circle.get_count())    # 2
```

3. Method Binding

Functions become bound methods when accessed through an instance

```
class Dog:
    def bark(self):
        print(f"{self.name} barks!")

d = Dog()
d.name = "Rex"

# Access through CLASS → plain function (unbound)
print(Dog.bark)          # <function Dog.bark at 0x...>
print(type(Dog.bark))    # <class 'function'>

# Access through INSTANCE → bound method
print(d.bark)             # <bound method Dog.bark of <Dog object>>
print(type(d.bark))       # <class 'method'>

# A bound method has the instance "baked in"
d.bark()                  # same as:
Dog.bark(d)               # explicitly passing instance
```

How binding works internally — Descriptors


```

# Functions are non-data descriptors – they implement __get__
# When you access d.bark, Python calls:
method = Dog.bark.__get__(d, Dog)
# This returns a bound method wrapping (Dog.bark, d)

# You can verify:
import inspect
print(d.bark.__self__)    # <Dog object> – the bound instance
print(d.bark.__func__)    # <function Dog.bark> – the original function

```

The three method types and their binding

```

class MyClass:
    class_var = 42

    def instance_method(self):          # receives instance
        return self

    @classmethod
    def class_method(cls):             # receives class
        return cls

    @staticmethod
    def static_method():               # receives nothing
        return "static"

obj = MyClass()

# Binding behavior:
obj.instance_method()    # self = obj (auto-bound)
obj.class_method()       # cls = MyClass (auto-bound)
obj.static_method()      # no binding at all

MyClass.instance_method(obj)    # must pass instance manually
MyClass.class_method()         # cls = MyClass (still auto-bound)
MyClass.static_method()        # same

```

classmethod vs staticmethod internals

```

# classmethod is a descriptor:
# classmethod.__get__(obj, cls) returns a bound method with cls as first arg

# staticmethod is a descriptor that SUPPRESSES binding:

```

```
# staticmethod.__get__(obj, cls) returns the raw function

class Demo:
    @classmethod
    def cm(cls): return cls

    @staticmethod
    def sm(): return "hi"

print(type(Demo.__dict__['cm']))    # <class 'classmethod'> - the descriptor
print(type(Demo.__dict__['sm']))    # <class 'staticmethod'> - the descriptor
print(type(Demo.cm))                # <class 'method'> - after __get__
print(type(Demo.sm))                # <class 'function'> - __get__ returns
raw fn
```

4. **self** Mechanics

self is not a keyword — it's just a convention

```
class Dog:
    def bark(this_dog):          # totally valid, just bad style
        print(f"{this_dog.name}!")

# Python doesn't care about the name - it's just the first parameter
# The CALLER automatically passes the instance as first argument
```

How **self** gets passed — the full chain

```
class Dog:
    def bark(self):
        print(id(self))

d = Dog()

# d.bark() internally does:
# 1. type(d).__mro__ lookup finds 'bark' in Dog.__dict__
# 2. Dog.__dict__['bark'].__get__(d, Dog) → bound method
# 3. bound_method() → calls Dog.bark(d)

print(id(d))          # same id
d.bark()               # same id - `self` IS `d`
```

`self` in inheritance

```
class Animal:
    def who_am_i(self):
        print(type(self))    # self is the ACTUAL instance type, not Animal

class Dog(Animal):
    pass

d = Dog()
d.who_am_i()    # <class '__main__.Dog'> - NOT Animal
                # self always refers to the actual runtime object
```

`self` and `__dict__` relationship

```
class Person:
    def __init__(self, name, age):
        self.name = name    # same as: self.__dict__['name'] = name
        self.age = age      # same as: self.__dict__['age'] = age

p = Person("Alice", 30)
print(p.__dict__)    # {'name': 'Alice', 'age': 30}

# You can manipulate __dict__ directly (generally avoid):
p.__dict__['email'] = 'alice@example.com'
print(p.email)    # alice@example.com
```

5. `__init__` vs `__new__`

The full object creation sequence

```
obj = MyClass(args)

# Internally Python does:
# Step 1: MyClass.__call__(args)          ← metaclass type's __call__
#   Step 1a: obj = MyClass.__new__(MyClass, args)  ← CREATE the object
#   Step 1b: if isinstance(obj, MyClass):
#               MyClass.__init__(obj, args)        ← INITIALIZE the object
# Step 2: return obj
```

`__new__` — the allocator

```

class MyClass:
    def __new__(cls, *args, **kwargs):
        print(f"__new__ called, cls={cls}")
        instance = super().__new__(cls)  # allocates memory, returns raw object
        return instance  # MUST return the instance

    def __init__(self, value):
        print(f"__init__ called, self={self}")
        self.value = value

obj = MyClass(42)
# __new__ called, cls=<class 'MyClass'>
# __init__ called, self=<MyClass object>

```

Key difference: `__new__` returns, `__init__` mutates

```

# __new__: creates and RETURNS the instance (can return different type!)
# __init__: receives the instance, sets it up, returns None

class Weird:
    def __new__(cls):
        print("__new__")
        return 42  # returning a non-instance!

    def __init__(self):
        print("__init__")  # NEVER called if __new__ doesn't return an instance

obj = Weird()
# __new__
# (no __init__)
print(obj)  # 42 ← just an int Practical use of __new__: Singleton pattern

```

```

class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance  # always returns the SAME object

    def __init__(self, value):

```

```

        self.value = value

s1 = Singleton(1)
s2 = Singleton(2)
print(s1 is s2)      # True – same object
print(s1.value)      # 2 – __init__ ran twice on same object!

```

Practical use of `__new__`: Immutable type subclassing

```

# You CANNOT modify immutable types in __init__ (object already exists)
# Must use __new__ to intercept creation

class UpperStr(str):
    def __new__(cls, value):
        return super().__new__(cls, value.upper()) # must transform before c
reation

s = UpperStr("hello")
print(s)          # HELLO
print(type(s))    # <class '__main__.UpperStr'>

# Same pattern for int, tuple, frozenset subclasses
class PositiveInt(int):
    def __new__(cls, value):
        if value < 0:
            raise ValueError("Must be positive")
        return super().__new__(cls, value)

```

`__init__` without `__new__` — the normal case

```

class Person:
    def __init__(self, name, age):
        # At this point, self already EXISTS (allocated by object.__new__)
        # __init__ just fills it with data
        if age < 0:
            raise ValueError("Age can't be negative")
        self.name = name
        self.age = age
        # implicitly returns None – Python ignores any return value

```

Calling order in inheritance

```

class Base:
    def __new__(cls, *a, **kw):
        print(f"Base.__new__, cls={cls.__name__}")
        return super().__new__(cls)

    def __init__(self, x):
        print(f"Base.__init__, x={x}")
        self.x = x

class Child(Base):
    def __new__(cls, *a, **kw):
        print(f"Child.__new__")
        return super().__new__(cls, *a, **kw)    # calls Base.__new__

    def __init__(self, x, y):
        print(f"Child.__init__")
        super().__init__(x)    # calls Base.__init__
        self.y = y

c = Child(1, 2)
# Child.__new__
# Base.__new__, cls=Child
# Child.__init__
# Base.__init__, x=1

```

6. Dunder (Magic) Methods — Complete Reference

`__repr__` vs `__str__`

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        # Goal: unambiguous, for developers, ideally eval()-able
        return f"Point({self.x!r}, {self.y!r})"

    def __str__(self):
        # Goal: readable, for end users
        return f"({self.x}, {self.y})"

p = Point(3, 4)

```

```

print(repr(p))    # Point(3, 4)    ← uses __repr__
print(str(p))     # (3, 4)         ← uses __str__
print(p)          # (3, 4)         ← print() uses __str__

# In collections, __repr__ is ALWAYS used:
print([p])        # [Point(3, 4)] ← list uses repr of elements

# If only __repr__ defined, it serves as fallback for __str__ too
# If only __str__ defined, repr() shows default <Point object at 0x...>

```

Comparison dunders

```

from functools import total_ordering

@total_ordering # auto-generates missing comparison methods from __eq__ + o
ne of __lt__/_gt__
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius

    def __eq__(self, other):
        if not isinstance(other, Temperature):
            return NotImplemented # let Python try the other operand
        return self.celsius == other.celsius

    def __lt__(self, other):
        if not isinstance(other, Temperature):
            return NotImplemented
        return self.celsius < other.celsius

    def __hash__(self):
        return hash(self.celsius) # MUST define if you define __eq__!
        # Python sets __hash__ = None if you define __eq__ without __hash__
        # making the object unhashable (can't be in sets/dicts)

t1 = Temperature(100)
t2 = Temperature(50)
print(t1 > t2)    # True (generated by @total_ordering)
print(t1 != t2)   # True (generated by @total_ordering)

```

__hash__ rules (CRITICAL for interviews)

```

# Rule 1: If a == b, then hash(a) == hash(b) MUST be true
# Rule 2: If you define __eq__, you MUST define __hash__

```

```
# Rule 3: Objects are unhashable if __hash__ is None

class BadClass:
    def __eq__(self, other): return True
    # __hash__ is now None – Python set it to None automatically!

b = BadClass()
# hash(b)           # TypeError: unhashable type
# {b}               # TypeError: unhashable type

# Mutable objects should be unhashable (list, dict have __hash__ = None)
# Immutable objects should be hashable (tuple, str, int)
```

Arithmetic dunders + reflected versions

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    # Normal operators
    def __add__(self, other):          # self + other
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, scalar):         # self * scalar (Vector * 3)
        return Vector(self.x * scalar, self.y * scalar)

    # Reflected (right-hand) operators – called when left operand fails
    def __rmul__(self, scalar):        # scalar * self (3 * Vector)
        return self.__mul__(scalar)

    # In-place operators
    def __imul__(self, scalar):        # self *= scalar
        self.x *= scalar
        self.y *= scalar
        return self                  # MUST return self

    def __neg__(self):                 # -self (unary)
        return Vector(-self.x, -self.y)

    def __abs__(self):                 # abs(self)
        return (self.x**2 + self.y**2) ** 0.5

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"
```



```

v = Vector(1, 2)
print(v + Vector(3, 4))    # Vector(4, 6)
print(v * 3)               # Vector(3, 6)
print(3 * v)               # Vector(3, 6) ← __rmul__
print(-v)                  # Vector(-1, -2)
print(abs(v))              # 2.23...

```

Reflected operator resolution

```

# For a + b:
# 1. Try type(a).__add__(a, b)
#    - If returns NotImplemented, go to step 2
# 2. Try type(b).__radd__(b, a)
#    - If returns NotImplemented, raise TypeError
# Special: if type(b) is a SUBCLASS of type(a), try __radd__ FIRST

```

Container dunders

```

class SizedList:
    def __init__(self, items):
        self._data = list(items)

    def __len__(self):          # len(obj)
        return len(self._data)

    def __getitem__(self, idx): # obj[idx], also enables iteration if no __i
        return self._data[idx]

    def __setitem__(self, idx, val): # obj[idx] = val
        self._data[idx] = val

    def __delitem__(self, idx):     # del obj[idx]
        del self._data[idx]

    def __contains__(self, item):   # item in obj
        return item in self._data

    def __iter__(self):             # for x in obj
        return iter(self._data)

    def __reversed__(self):         # reversed(obj)
        return reversed(self._data)

```

```
sl = SizedList([1, 2, 3, 4])
print(len(sl))      # 4
print(sl[1])        # 2
print(3 in sl)      # True
sl[0] = 99
del sl[-1]
```

Attribute access dunders

```
class Logged:
    def __getattr__(self, name):
        # Called ONLY when normal lookup FAILS (not in __dict__ or class)
        print(f"Getting missing attribute: {name}")
        raise AttributeError(name)

    def __getattribute__(self, name):
        # Called for EVERY attribute access (use with extreme care)
        print(f"Accessing: {name}")
        return super().__getattribute__(name) # MUST call super or infinite
        recursion

    def __setattr__(self, name, value):
        # Called for EVERY attribute assignment
        print(f"Setting {name} = {value}")
        super().__setattr__(name, value) # MUST call super

    def __delattr__(self, name):
        print(f"Deleting {name}")
        super().__delattr__(name)
```

`__getattr__` vs `__getattribute__`

```
class Demo:
    def __init__(self):
        self.real = "I exist"

    def __getattr__(self, name):
        # Only called when attribute NOT found normally
        return f"default_{name}"

    def __getattribute__(self, name):
        # Called for EVERY access – real AND missing attributes
        return super().__getattribute__(name)
```

```
d = Demo()
print(d.real)      # "I exist" – found in __dict__, __getattr__ NOT called
print(d.fake)     # "default_fake" – not found → __getattr__ called
```

Context manager dunders

```
class ManagedResource:
    def __enter__(self):
        print("Acquiring resource")
        return self          # value bound to `as` variable

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Releasing resource")
        if exc_type is ValueError:
            print("Suppressing ValueError")
            return True       # suppress the exception
        return False         # re-raise any other exception

with ManagedResource() as r:
    raise ValueError("oops")  # suppressed!
# Acquiring resource
# Releasing resource
# Suppressing ValueError
```

Callable dunders

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, value):    # makes instance callable like a function
        return value * self.factor

double = Multiplier(2)
print(double(5))                # 10
print(callable(double))        # True
```

Object lifecycle dunders

```
class Tracked:
    _live = 0
```

```

def __new__(cls, *args, **kwargs):
    cls._live += 1
    return super().__new__(cls)

def __del__(self):
    # Called when garbage collected (NOT guaranteed timing)
    type(self)._live -= 1
    print(f"Object destroyed, {type(self)._live} remaining")
    # WARNING: avoid complex logic here, __del__ has many pitfalls

```

Copy dunders

```

import copy

class DeepObj:
    def __copy__(self):
        # Called by copy.copy() – shallow copy
        new = type(self).__new__(type(self))
        new.__dict__.update(self.__dict__)
        return new

    def __deepcopy__(self, memo):
        # Called by copy.deepcopy()
        # memo dict prevents infinite recursion on circular refs
        new = type(self).__new__(type(self))
        memo[id(self)] = new
        for k, v in self.__dict__.items():
            setattr(new, k, copy.deepcopy(v, memo))
        return new

```

Pickling dunders

```

class Serializable:
    def __getstate__(self):
        # Return state to be pickled (can exclude unpicklable attrs)
        state = self.__dict__.copy()
        del state['unpicklable_connection']
        return state

    def __setstate__(self, state):
        # Restore state when unpickling
        self.__dict__.update(state)
        self.unpicklable_connection = reconnect() # re-create

```

7. MRO & C3 Linearization

The problem MRO solves — The Diamond Problem

```
class A:
    def method(self): print("A")

class B(A):
    def method(self): print("B")

class C(A):
    def method(self): print("C")

class D(B, C):    # inherits from both B and C
    pass

d = D()
d.method()      # Which method? B or C or A?
                # Answer: B — because of MRO
```

MRO resolution order

```
print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
# D → B → C → A → object

# Python searches in this EXACT order, uses FIRST match found
```

C3 Linearization algorithm

The MRO of class C with bases B1, B2, ... is computed as:

```
L[C] = C + merge(L[B1], L[B2], ..., [B1, B2, ...])

merge algorithm:
1. Take the first element of the first list (head)
2. Check if it appears in the TAIL of any other list
3. If NOT → add to result, remove from all lists
4. If YES → skip, try head of next list
5. Repeat until all lists empty (or raise TypeError if impossible)
```

```

# Example: class D(B, C) where B(A), C(A)
# L[A]      = [A, object]
# L[B]      = [B, A, object]
# L[C]      = [C, A, object]
# L[D]      = D + merge([B, A, object], [C, A, object], [B, C])

# Step 1: head=B, in tail of [B,C]? NO → take B
#         merge([A, object], [C, A, object], [C])
# Step 2: head=A, in tail of [C,A,object]? YES → skip
#         head=C, in tail of [C]? NO → take C
#         merge([A, object], [A, object], [])
# Step 3: head=A, in tail of []? NO → take A
#         merge([object], [object], [])
# Step 4: take object
# Result: D → B → C → A → object ✓

```

MRO with a contradictory hierarchy (TypeError)

```

class X: pass
class Y: pass
class A(X, Y): pass
class B(Y, X): pass

# class C(A, B): pass    # TypeError! Cannot create consistent MRO
# A says X before Y, B says Y before X – contradiction

```

super() and MRO — how super REALLY works

```

class A:
    def greet(self):
        print("A.greet")

class B(A):
    def greet(self):
        print("B.greet")
        super().greet()    # NOT "call A.greet" – calls NEXT in MRO

class C(A):
    def greet(self):
        print("C.greet")
        super().greet()

class D(B, C):

```

```

def greet(self):
    print("D.greet")
    super().greet()

D().greet()
# D.greet
# B.greet      ← super() in D follows MRO: next after D is B
# C.greet      ← super() in B follows MRO: next after B is C (NOT A!)
# A.greet      ← super() in C follows MRO: next after C is A

```

`super()` internals

```

# super() with no arguments (Python 3) is equivalent to:
# super(__class__, self)

# super(type, obj).__getattr__(name) does:
# 1. Take obj's MRO
# 2. Find `type` in that MRO
# 3. Search from the NEXT class onwards

class B(A):
    def method(self):
        # These are equivalent:
        super().method()
        super(B, self).method()

        # Super with class (not instance) for classmethods:
        super(B, B).classmethod()

```

8. Multiple Inheritance

Patterns and pitfalls

```

class Flyable:
    def fly(self): print("flying")
    def move(self): print("Flyable.move")

class Swimmable:
    def swim(self): print("swimming")
    def move(self): print("Swimmable.move")

class Duck(Flyable, Swimmable):
    pass

```

```

d = Duck()
d.fly()      # flying
d.swim()     # swimming
d.move()     # Flyable.move ← first in MRO wins
print(Duck.__mro__) # Duck → Flyable → Swimmable → object

```

Cooperative multiple inheritance with `super()`

```

# For super() to work cooperatively in MI, EVERY class must use super()

class Base:
    def __init__(self):
        print("Base.__init__")
        # No super().__init__() needed here (would call object.__init__)

class Mixin1:
    def __init__(self):
        print("Mixin1.__init__")
        super().__init__() # passes control to next in MRO

class Mixin2:
    def __init__(self):
        print("Mixin2.__init__")
        super().__init__() # passes control to next in MRO

class MyClass(Mixin1, Mixin2, Base):
    def __init__(self):
        print("MyClass.__init__")
        super().__init__() # starts the cooperative chain

MyClass()
# MyClass.__init__
# Mixin1.__init__
# Mixin2.__init__
# Base.__init__
# Each __init__ called EXACTLY ONCE despite multiple inheritance

```

`__init__` argument passing in cooperative MI

```

# Problem: different __init__ signatures in MI chain
# Solution: use **kwargs and absorb what you need

```

```

class A:

```



```

def __init__(self, a, **kwargs):
    print(f"A: a={a}")
    super().__init__(**kwargs)

class B:
    def __init__(self, b, **kwargs):
        print(f"B: b={b}")
        super().__init__(**kwargs)

class C(A, B):
    def __init__(self, a, b):
        super().__init__(a=a, b=b)

C(a=1, b=2)
# A: a=1
# B: b=2

```

9. Mixins

What is a Mixin?

A **Mixin** is a class designed to provide methods to other classes through multiple inheritance — NOT meant to be instantiated on its own, adds a specific capability.

```

class SerializeMixin:
    """Adds JSON serialization to any class"""
    def to_dict(self):
        return self.__dict__

    def to_json(self):
        import json
        return json.dumps(self.to_dict())

    @classmethod
    def from_dict(cls, data):
        obj = cls.__new__(cls)
        obj.__dict__.update(data)
        return obj

class LogMixin:
    """Adds logging to any class"""
    def log(self, msg):
        import logging
        logging.getLogger(type(self).__name__).info(msg)

```

```

class ValidateMixin:
    """Adds validation hooks"""
    def validate(self):
        for attr, validator in getattr(self, '_validators', {}).items():
            if not validator(getattr(self, attr)):
                raise ValueError(f"Validation failed for {attr}")

class User(SerializeMixin, LogMixin, ValidateMixin):
    _validators = {
        'age': lambda x: x > 0,
        'name': lambda x: len(x) > 0
    }

    def __init__(self, name, age):
        self.name = name
        self.age = age

u = User("Alice", 30)
print(u.to_json())      # {"name": "Alice", "age": 30}
u.log("User created")
u.validate()

```

Mixin design rules

```

# Rule 1: Mixin should NOT have __init__ (or use **kwargs if it must)
# Rule 2: Mixin should not inherit from the class it's mixed into
# Rule 3: Mixin names should end in 'Mixin' or start with a verb
# Rule 4: Mixins should be listed BEFORE the main class in bases

class MyModel(CacheMixin, SerializeMixin, LogMixin, BaseModel):
#                                     ^^^^^^^^^^^
#                                     main class goes last
    pass

```

Mixin with abstract-like requirements

```

class ComparableMixin:
    """Requires subclass to implement __lt__"""
    def __gt__(self, other): return other.__lt__(self)
    def __le__(self, other): return self.__lt__(other) or self == other
    def __ge__(self, other): return not self.__lt__(other)

class Temperature(ComparableMixin):

```

```

def __init__(self, degrees):
    self.degrees = degrees

def __lt__(self, other):          # required by ComparableMixin
    return self.degrees < other.degrees

def __eq__(self, other):
    return self.degrees == other.degrees

t1 = Temperature(100)
t2 = Temperature(50)
print(t1 > t2)    # True ← provided by mixin
print(t1 <= t2)   # False ← provided by mixin

```

10. Composition vs Inheritance

Inheritance — "is-a" relationship

```

class Animal:
    def breathe(self): print("breathing")

class Dog(Animal):          # Dog IS-A Animal
    def bark(self): print("woof")

# ✅ Good: Dog genuinely IS an Animal, substitutable everywhere Animal is used
# Liskov Substitution Principle: subclass must be usable wherever parent is used

```

Composition — "has-a" relationship

```

class Engine:
    def start(self): print("vroom")
    def stop(self): print("off")

class Car:                  # Car HAS-A Engine (not IS-A Engine)
    def __init__(self):
        self.engine = Engine()  # composition

    def drive(self):
        self.engine.start()
        print("driving")

```

```
# ✅ Good: Car is not a type of Engine – it USES an engine
```

Why "favor composition over inheritance"

```
# Inheritance problem 1: Fragile base class
class List:
    def append(self, item): ...
    def extend(self, items):
        for item in items:
            self.append(item)    # calls self.append

class CountingList(List):
    def __init__(self):
        self.count = 0

    def append(self, item):
        self.count += 1
        super().append(item)

cl = CountingList()
cl.extend([1, 2, 3])
print(cl.count)    # 3 – works now, but base class internals can change!

# Inheritance problem 2: Wrong abstraction forces awkward overrides
class Rectangle:
    def set_width(self, w): self.width = w
    def set_height(self, h): self.height = h
    def area(self): return self.width * self.height

class Square(Rectangle):    # LSP VIOLATION
    def set_width(self, w):
        self.width = w
        self.height = w    # forced to sync both

    def set_height(self, h):
        self.height = h
        self.width = h

r = Rectangle()
r.set_width(4)
r.set_height(5)
print(r.area())    # 20 ✓
```

```
s = Square()
s.set_width(4)
s.set_height(5)    # breaks the square contract!
print(s.area())    # 25 – surprising!
```

Composition solution

```
class Shape:
    pass

class Rectangle(Shape):
    def __init__(self, w, h):
        self.width = w
        self.height = h
    def area(self): return self.width * self.height

class Square(Shape):          # NOT a Rectangle
    def __init__(self, side):
        self.side = side
    def area(self): return self.side ** 2

# Both are Shapes but neither inherits from the other – no LSP issue
```

Practical comparison

```
# ❌ Deep inheritance – hard to follow, brittle
class Vehicle: pass
class MotorVehicle(Vehicle): pass
class Car(MotorVehicle): pass
class ElectricCar(Car): pass
class TeslaModelS(ElectricCar): pass    # 5 levels deep!

# ✅ Composition – flat, flexible, testable
class ElectricMotor:
    def power_output(self): return 400    # kW

class Battery:
    def charge_level(self): return 0.8    # 80%

class AutopilotSystem:
    def navigate(self): pass

class TeslaModelS:
    def __init__(self):
```

```
self.motor = ElectricMotor()
self.battery = Battery()
self.autopilot = AutopilotSystem()
# Swap any component without touching others!
```

When to use each

```
# USE INHERITANCE when:
# ✓ Clear is-a relationship (Dog is an Animal)
# ✓ Subclass passes the Liskov Substitution test
# ✓ You need to override behavior polymorphically
# ✓ Sharing interface (ABCs)

# USE COMPOSITION when:
# ✓ Has-a or uses-a relationship (Car has an Engine)
# ✓ Behavior may change at runtime (strategy pattern)
# ✓ Combining behaviors from multiple unrelated classes
# ✓ Avoiding tight coupling to parent implementation

# USE MIXINS when:
# ✓ Adding a capability orthogonal to the hierarchy
# ✓ The capability is reusable across unrelated classes
# ✓ No state needed (or minimal state)
```

Interview Cheat Sheet

```
__new__ → creates object (allocator), returns instance
__init__ → initializes object, returns None

__repr__ → unambiguous, for devs (fallback for __str__)
__str__ → readable, for users

__hash__ → MUST define if __eq__ defined (or set = None)
__eq__ → MUST be consistent with __hash__

super() → next in MRO, not necessarily parent class
MRO → C3 linearization, D→B→C→A→object for diamond

Mixin → capability class, not instantiated alone
Composition → has-a, prefer over deep inheritance

self → just a convention, auto-passed bound instance
cls → auto-passed by @classmethod descriptor
```

```
class creation: metaclass.__prepare__ → exec body → type()
```

Python Exception Handling Internals — Complete Interview Notes

1. Exception Hierarchy

The complete built-in tree

```
BaseException
├── SystemExit          ← sys.exit() raises this
├── KeyboardInterrupt  ← Ctrl+C raises this
├── GeneratorExit       ← generator.close() raises this
└── Exception          ← ALL normal exceptions inherit from here
    ├── StopIteration  ← end of iterator
    ├── StopAsyncIteration ← end of async iterator
    ├── ArithmeticError
    │   ├── FloatingPointError
    │   ├── OverflowError
    │   └── ZeroDivisionError
    ├── AssertionError  ← assert statement failed
    ├── AttributeError  ← obj.attr doesn't exist
    ├── BufferError
    ├── EOFError
    ├── ImportError
    │   └── ModuleNotFoundError
    ├── LookupError
    │   ├── IndexError  ← list[99] out of range
    │   └── KeyError     ← dict['missing']
    ├── MemoryError
    ├── NameError
    │   └── UnboundLocalError
    ├── OSError (IOError alias)
    │   ├── FileNotFoundError
    │   ├── PermissionError
    │   ├── TimeoutError
    │   ├── ConnectionError
    │   │   ├── BrokenPipeError
    │   │   ├── ConnectionAbortedError
    │   │   └── ConnectionRefusedError
```

```

|   |   └─ ConnectionResetError
|   └─ FileNotFoundError
└─ ReferenceError
└─ RuntimeError
    │   └─ NotImplementedError ← abstract method not implemented
    │   └─ RecursionError
└─ SyntaxError
    │   └─ IndentationError
    │       └─ TabError
└─ SystemError
└─ TypeError
└─ ValueError
    │   └─ UnicodeError
    │       │   └─ UnicodeDecodeError
    │       │   └─ UnicodeEncodeError
    │       └─ UnicodeTranslateError
└─ Warning
    │   └─ DeprecationWarning
    │   └─ RuntimeWarning
    │   └─ SyntaxWarning
    │   └─ UserWarning
    └─ FutureWarning

```

Why `BaseException` exists separately from `Exception`

```

# BaseException catches EVERYTHING – almost never what you want
try:
    sys.exit(0)
except BaseException as e:
    print(f"Caught: {e}") # catches SystemExit – bad practice!

# Exception catches all normal errors – what you almost always want
try:
    risky_operation()
except Exception as e:
    print(f"Caught: {e}") # will NOT catch SystemExit, KeyboardInterrupt

# Critical rule: NEVER do bare except: or except BaseException:
# unless you immediately re-raise or have a very specific reason
try:
    risky()
except:
    # ← catches EVERYTHING including KeyboardInterrupt!
    pass
    # ← swallows Ctrl+C silently – very bad

```


Exception IS-A relationships — catching parent catches children

```
try:
    d = {}
    d['missing']          # raises KeyError
except LookupError as e:  # catches KeyError because KeyError IS-A LookupError
    or
    print(type(e))        # <class 'KeyError'>
    print(isinstance(e, LookupError))  # True
    print(isinstance(e, Exception))    # True
    print(isinstance(e, BaseException)) # True

# Multiple except clauses – ORDER MATTERS (most specific first)
try:
    x = int("abc")
except ValueError:
    print("value error")
except Exception:
    print("some error")    # never reached for ValueError – already caught above

# ❌ Wrong order – specific exception can never be reached
try:
    x = [[]][99]
except Exception:         # catches IndexError here
    print("generic")
except IndexError:        # DEAD CODE – IndexError already matched above
    print("index")        # never reached
```

2. Exception Objects Internals

What an exception object IS

```
# All exceptions are class instances
e = ValueError("bad value")

print(type(e))          # <class 'ValueError'>
print(e.args)           # ('bad value',) ← tuple of constructor args
print(str(e))           # 'bad value'    ← calls __str__
print(repr(e))          # ValueError('bad value')

# Multiple args
e2 = OSError(2, "No such file", "/tmp/x")
```

```

print(e2.args)          # (2, 'No such file', '/tmp/x')
print(e2.errno)         # 2          ← OSError-specific attribute
print(e2.strerror)      # 'No such file'
print(e2.filename)      # '/tmp/x'

```

Exception attributes set during raise

```

import traceback, sys

try:
    1 / 0
except ZeroDivisionError as e:
    print(e.__traceback__)    # traceback object
    print(e.__class__)        # <class 'ZeroDivisionError'>
    print(e.__cause__)        # None (no explicit chaining)
    print(e.__context__)      # None (no implicit chaining here)
    print(e.__suppress_context__) # False

    # Get full traceback as string:
    tb_str = traceback.format_exc()
    print(tb_str)

```

3. Exception Propagation

The call stack unwinding model

```

def level3():
    raise ValueError("deep error")    # ← exception BORN here

def level2():
    level3()                          # ← propagates UP (no handler)

def level1():
    level2()                          # ← propagates UP (no handler)

def main():
    try:
        level1()                    # ← caught HERE
    except ValueError as e:
        print(f"Caught: {e}")

main()

```

Frame-by-frame propagation — what Python does internally

1. `level3()` raises `ValueError`
 - Python creates exception object
 - Python checks `level3`'s frame for matching except clause
 - `NONE` found → Python UNWINDS `level3`'s frame (pops it from call stack)
 - Exception propagates to `level2`'s frame
2. `level2()` has no handler
 - Frame unwound → propagates to `level1`'s frame
3. `level1()` has no handler
 - Frame unwound → propagates to `main`'s frame
4. `main()` has matching except `ValueError`
 - Exception CAUGHT → execution resumes after except block
 - All unwound frames are gone forever

Propagation through multiple except clauses

```
try:
    raise IndexError("bad index")
except ValueError:
    print("ValueError")    # skipped – doesn't match
except KeyError:
    print("KeyError")      # skipped – doesn't match
except (IndexError, TypeError):
    print("IndexError or TypeError")  # ✓ MATCHES – caught here
except Exception:
    print("never reached") # skipped – already caught above
```

Exception propagation and `else`

```
def divide(a, b):
    try:
        result = a / b          # might raise ZeroDivisionError
    except ZeroDivisionError:
        print("Cannot divide by zero")
        return None
    else:
        # ONLY runs if NO exception was raised in try block
        # NOT the same as putting code at end of try!
        print("Division successful")
        return result
```

```

    finally:
        # ALWAYS runs – exception or not, return or not
        print("divide() finished")

divide(10, 2)
# Division successful      ← else ran
# divide() finished       ← finally ran

divide(10, 0)
# Cannot divide by zero   ← except ran (else did NOT run)
# divide() finished       ← finally ran

```

4. Stack Unwinding — Deep Internals

What "unwinding" means at the bytecode level

```

import dis

def example():
    try:
        risky()
    except ValueError:
        handle()

dis.dis(example)
# Key bytecode instructions:
# SETUP_FINALLY → pushes exception handler onto the block stack
# POP_BLOCK     → removes handler when try exits normally
# JUMP_FORWARD  → skips except block after normal execution
# With exception: bytecode jumps to except handler address

```

The block stack — Python's internal exception machinery

```

# Python maintains a "block stack" alongside the call stack
# Each try/except/finally pushes an entry onto the block stack

def nested_try():
    try:                                     # block stack: [handler_A]
        try:                               # block stack: [handler_A, handler_B]
            raise ValueError("x")
        except TypeError:                  # handler_B doesn't match
            pass                           # block stack: [handler_A]

```

```
except ValueError:           # handler_A matches ✓
    print("caught")          # block stack: []
```

Finally and unwinding — finally ALWAYS runs

```
def always_runs():
    try:
        print("try")
        return "from try"      # finally still runs even with return!
    finally:
        print("finally")       # runs before return completes
        # return "from finally" # ← this would OVERRIDE "from try"!

result = always_runs()
# try
# finally
print(result)  # "from try"
```

finally overriding return/exception — the trap

```
def trap1():
    try:
        return 1
    finally:
        return 2              # △ OVERRIDES try's return!

print(trap1())  # 2 ← finally's return wins

def trap2():
    try:
        raise ValueError("original")
    finally:
        return 42             # △ SILENTLY SWALLOWS the exception!

print(trap2())  # 42 ← exception completely lost!

def trap3():
    try:
        raise ValueError("original")
    finally:
        raise RuntimeError("new") # △ original exception REPLACED
        # original ValueError is lost – chained as __context__ though
```

Full unwinding sequence — every scenario

```
def full_demo():
    try:          # ← 1. enter try block
        pass     # ← 2. execute try body
    except E:    # ← 3a. only if exception matches E
        pass    # ← 4a. execute except body
    else:       # ← 3b. only if NO exception was raised
        pass    # ← 4b. execute else body
    finally:    # ← 5. ALWAYS executes
        pass   # ← 6. execute finally body

# Execution paths:
# No exception:    try → else → finally
# Matching except: try → except → finally
# Non-matching:    try → finally → propagate upward
# return in try:   try → finally → return
# return in except: try → except → finally → return
```

5. Custom Exceptions

Basic custom exception

```
class AppError(Exception):
    """Base exception for this application"""
    pass

class DatabaseError(AppError):
    """All database-related errors"""
    pass

class ConnectionError(DatabaseError):
    """Database connection failed"""
    pass

class QueryError(DatabaseError):
    """Query execution failed"""
    pass

# Catching at different levels:
try:
    raise QueryError("syntax error in SQL")
except QueryError:
```

```

    print("handle specific query error")    # most specific

try:
    raise ConnectionError("timeout")
except DatabaseError:
    print("handle any DB error")           # catches both ConnectionError and
    QueryError

try:
    raise DatabaseError("unknown")
except AppError:
    print("handle any app error")          # catches all app errors

```

Custom exception with rich context

```

class ValidationError(Exception):
    def __init__(self, field, value, message):
        self.field = field
        self.value = value
        self.message = message
        # Pass useful string to parent for str(e) and logging
        super().__init__(f"Validation failed for '{field}': {message} (got {value!r})")

    def __repr__(self):
        return (f"ValidationError(field={self.field!r}, "
                f"value={self.value!r}, message={self.message!r})")

try:
    raise ValidationError("age", -5, "must be positive")
except ValidationError as e:
    print(e)           # Validation failed for 'age': must be positive (got -5)

    print(e.field)     # age
    print(e.value)     # -5
    print(repr(e))     # ValidationError(field='age', value=-5, message='must
                        # be positive')

```

Exception hierarchy for a real application

```

# ✅ Best practice: one base exception per package/app
class PlatformError(Exception):
    """Root exception – catch-all for this platform"""
    def __init__(self, message, code=None):

```

```

        super().__init__(message)
        self.code = code

class AuthError(PlatformError):
    """Authentication/authorization failures"""
    pass

class TokenExpiredError(AuthError):
    def __init__(self):
        super().__init__("Token has expired", code="TOKEN_EXPIRED")

class PermissionDeniedError(AuthError):
    def __init__(self, resource):
        super().__init__(f"Access denied to {resource!r}", code="PERMISSION_DENIED")
        self.resource = resource

class NetworkError(PlatformError):
    def __init__(self, url, status_code):
        super().__init__(f"Request to {url} failed with {status_code}", code="NETWORK_ERROR")
        self.url = url
        self.status_code = status_code

# Usage:
try:
    raise PermissionDeniedError("/admin")
except AuthError as e:
    print(f"Auth issue [{e.code}]: {e}")
except PlatformError as e:
    print(f"Platform issue [{e.code}]: {e}")

```

Exception with `__reduce__` for pickling

```

class PicklableError(Exception):
    def __init__(self, message, detail):
        super().__init__(message)
        self.detail = detail

    def __reduce__(self):
        # needed when custom __init__ has extra args
        return (self.__class__, (str(self), self.detail))

import pickle

```



```
e = PicklableError("oops", {"key": "value"})
e2 = pickle.loads(pickle.dumps(e))
print(e2.detail)    # {'key': 'value'}
```

6. `raise`, `raise from`, and Exception Chaining

Basic `raise` forms

```
# Form 1: raise a new exception
raise ValueError("bad value")

# Form 2: raise an existing exception instance
e = ValueError("bad value")
raise e

# Form 3: re-raise current exception (inside except only)
try:
    risky()
except ValueError:
    log_error()
    raise          # re-raises the SAME exception with SAME traceback

# Form 4: raise exception class (Python instantiates with no args)
raise ValueError  # equivalent to: raise ValueError()
```

Implicit exception chaining (`__context__`)

```
# When exception B is raised WHILE handling exception A,
# Python automatically sets B.__context__ = A

try:
    open("nonexistent.txt")          # raises FileNotFoundError
except FileNotFoundError:
    raise RuntimeError("setup failed")  # raised while handling FileNotFoundError

# Output:
# During handling of the above exception, another exception occurred:
# RuntimeError: setup failed
#
# B.__context__ = FileNotFoundError (set automatically by Python)
# B.__suppress_context__ = False (so context is shown)
```

Explicit chaining with `raise from (__cause__)`

```
# raise X from Y → sets X.__cause__ = Y and X.__suppress_context__ = True

try:
    int("abc") # raises ValueError
except ValueError as e:
    raise RuntimeError("parse failed") from e # explicit chain

# Output:
# ValueError: invalid literal for int()...
# The above exception was the DIRECT CAUSE of the following exception:
# RuntimeError: parse failed
#
# __cause__ = ValueError ← your explicit chain
# __suppress_context__ = True ← hides implicit context
```

Suppressing context with `raise from None`

```
# raise X from None → hides ALL exception context from traceback

try:
    int("abc")
except ValueError:
    raise RuntimeError("invalid input") from None

# Output: ONLY shows RuntimeError – ValueError completely hidden
# __cause__ = None
# __suppress_context__ = True

# Use this when the original exception is an implementation detail
# the caller shouldn't see (e.g., internal DB errors mapped to app errors)
```

Full chaining attribute map

```
try:
    try:
        raise ValueError("original")
    except ValueError as e:
        raise RuntimeError("wrapper") from e
except RuntimeError as e:
    print(e.__cause__) # ValueError: original (explicit)
    print(e.__context__) # ValueError: original (also set)
    print(e.__suppress_context__) # True (from sets this to True)
```

```

    print(e.__traceback__)          # traceback object

# Traversing the chain:
def print_chain(exc):
    if exc is None:
        return
    print(f"Exception: {exc!r}")
    if exc.__cause__:
        print("  CAUSED BY:")
        print_chain(exc.__cause__)
    elif exc.__context__ and not exc.__suppress_context__:
        print("  DURING HANDLING OF:")
        print_chain(exc.__context__)

```

7. **try / except / else / finally** — Every Scenario

Complete block semantics

```

def complete_example(data):
    resource = None
    try:
        # ① TRY: code that might fail
        resource = acquire_resource()
        result = process(data)

    except TypeError as e:
        # ② EXCEPT: runs ONLY if matching exception raised in try
        # `e` is bound here, deleted after except block exits
        print(f"Type error: {e}")
        result = default_result()

    except (ValueError, KeyError) as e:
        # Multiple exception types in one clause
        print(f"Value/Key error: {e}")
        raise    # re-raise after logging

    except Exception as e:
        # Catch-all for unexpected errors
        print(f"Unexpected: {e}")
        raise    # always re-raise unexpected errors!

    else:
        # ③ ELSE: runs ONLY if try completed WITHOUT any exception

```

```

    # Reason to use else: keep try block minimal,
    # separate "might fail" from "only runs on success"
    print(f"Success: {result}")
    save_result(result)    # if this fails, except blocks WON'T catch it

finally:
    # ④ FINALLY: ALWAYS runs, exception or not, return or not
    # Use for cleanup that MUST happen
    if resource:
        resource.release()

return result

```

else — the most misunderstood clause

```

# WHY use else instead of putting code at end of try?

# ❌ Without else – cleanup accidentally caught by except
try:
    result = risky_operation()
    save(result)          # if save() raises ValueError, except catches it!
except ValueError:
    handle()

# ✅ With else – save() errors propagate naturally
try:
    result = risky_operation()
except ValueError:
    handle()
else:
    save(result)          # ValueError here propagates UP, not to except

# else also communicates intent clearly:
# "this code only makes sense if the try succeeded"

```

Nested try/except — exception routing

```

def nested():
    try:                                # outer try
        try:                            # inner try
            raise ValueError("inner")
        except TypeError:
            print("inner except – no match, propagates out")
    # ValueError not caught → propagates to outer try

```

```

        except ValueError as e:                # caught here
            print(f"outer caught: {e}")

nested()
# outer caught: inner

```

Re-raise patterns — all variants

```

# Pattern 1: Log and re-raise same exception
try:
    operation()
except Exception as e:
    logger.error(f"Failed: {e}", exc_info=True) # includes traceback in log
    raise    # ← bare raise preserves original traceback

# Pattern 2: Wrap in application exception
try:
    db.query(sql)
except psycopg2.DatabaseError as e:
    raise QueryError("Query failed") from e    # chain explicitly

# Pattern 3: Conditional re-raise
try:
    result = parse(data)
except ValueError as e:
    if is_recoverable(e):
        result = default
    else:
        raise    # re-raise unrecoverable cases

# Pattern 4: Re-raise with added context
try:
    process(item)
except Exception as e:
    e.args = (f"Failed processing item {item!r}: {e.args[0]}",) + e.args[1:]
    raise    # same exception type, richer message

```

finally execution guarantees — all edge cases

```

# Case 1: Normal execution
try:
    pass
finally:

```

```

    print("runs")    # ✓

# Case 2: Exception caught
try:
    raise ValueError
except ValueError:
    pass
finally:
    print("runs")    # ✓

# Case 3: Exception NOT caught
try:
    raise ValueError
finally:
    print("runs")    # ✓ runs BEFORE exception propagates
# ValueError propagates after finally

# Case 4: return in try
def f():
    try:
        return 1
    finally:
        print("runs")    # ✓ runs before return completes
f()    # prints "runs", returns 1

# Case 5: sys.exit()
import sys
try:
    sys.exit(1)    # raises SystemExit
finally:
    print("runs")    # ✓ even sys.exit() triggers finally

# Case 6: os._exit() – bypasses ALL Python cleanup
import os
try:
    os._exit(1)    # ← raw OS exit, finally does NOT run
finally:
    print("never runs")

```

8. Cost of Exceptions

The zero-cost try model (CPython)

```

# In CPython, the TRY BLOCK ITSELF has near-zero cost
# Cost is paid only when exception is RAISED and CAUGHT

import timeit

def with_try_no_exception():
    try:
        x = 1 + 1      # no exception raised
    except ValueError:
        pass

def without_try():
    x = 1 + 1          # no try at all

# Timing:
t1 = timeit.timeit(with_try_no_exception, number=10_000_000)
t2 = timeit.timeit(without_try, number=10_000_000)
print(f"with try (no exception): {t1:.2f}s")
print(f"without try: {t2:.2f}s")
# Difference is negligible – try block setup is ~free in Python 3.11+

```

Where the REAL cost is — exception raising

```

import timeit

def using_exception():
    for _ in range(1000):
        try:
            {}['key']      # raises KeyError
        except KeyError:
            pass

def using_get():
    for _ in range(1000):
        {}.get('key')      # no exception

t1 = timeit.timeit(using_exception, number=10_000)
t2 = timeit.timeit(using_get, number=10_000)

print(f"exception path: {t1:.3f}s")
print(f"dict.get path: {t2:.3f}s")
# Exception path is 5-10x slower – traceback creation is expensive

```

What makes raising expensive

```
# When raise executes, Python must:
# 1. Instantiate the exception object
# 2. CAPTURE the full traceback (walk the call stack)
# 3. Attach __traceback__ frames (one per stack frame)
# 4. Unwind the block stack
# 5. Search for matching except clause (may walk many frames)

# The deeper the call stack, the more expensive the traceback capture

def deep(n):
    if n == 0:
        raise ValueError("deep")
    deep(n - 1)

import timeit

def shallow():
    try: deep(1)
    except ValueError: pass

def very_deep():
    try: deep(100)
    except ValueError: pass

t1 = timeit.timeit(shallow, number=100_000)
t2 = timeit.timeit(very_deep, number=100_000)
print(f"shallow (depth 1):  {t1:.3f}s")
print(f"deep    (depth 100): {t2:.3f}s")
# deeper stack = more expensive exception
```

LBYL vs EAFP — Python's philosophy

```
# LBYL – Look Before You Leap (check first, act second)
# Common in C, Java

def get_value_lbyl(d, key):
    if key in d:          # check (one dict lookup)
        return d[key]     # act   (another dict lookup) – 2 lookups total!
    return None

# EAFP – Easier to Ask Forgiveness than Permission (Python style)
def get_value_eafp(d, key):
```



```

try:
    return d[key]      # act (one lookup)
except KeyError:      # forgiveness (only paid if key missing)
    return None

# EAFP is more Pythonic and often faster when the "happy path" is common
# because it avoids the double-lookup

# LBYL is better when exceptions are FREQUENT (high miss rate)
# because exception overhead exceeds the double-lookup cost

# Race condition: LBYL has TOCTOU (Time-Of-Check-Time-Of-Use) problem
import os
# ❌ LBYL – race condition
if os.path.exists(filename):      # file exists here
    open(filename)                # deleted between check and open!

# ✅ EAFP – atomic
try:
    open(filename)
except FileNotFoundError:
    pass

```

Avoid exceptions in tight loops — practical optimization

```

# ❌ Exception in hot loop – slow
def sum_values_bad(items):
    total = 0
    for item in items:
        try:
            total += int(item)
        except (ValueError, TypeError):
            pass    # exception overhead on every bad item
    return total

# ✅ Check first in hot loop – fast when bad items are common
def sum_values_good(items):
    total = 0
    for item in items:
        if isinstance(item, (int, float)):
            total += item
        elif isinstance(item, str) and item.isdigit():
            total += int(item)
    return total

```

```
#  Best – use exceptions only for truly exceptional cases
def sum_values_best(items):
    # if bad items are RARE, EAFP is fine
    total = 0
    for item in items:
        try:
            total += item          # fast path – no exception overhead usually
        except TypeError:
            pass                  # rare path – exception cost is acceptable
    return total
```

Suppressing exceptions efficiently — `contextlib.suppress`

```
from contextlib import suppress

#  Verbose try/except for simple suppression
try:
    os.remove(temp_file)
except FileNotFoundError:
    pass

#  Clean and readable
with suppress(FileNotFoundError):
    os.remove(temp_file)

# suppress is implemented as a context manager – same cost as try/except
# but cleaner for "I don't care if this fails" scenarios

# Multiple exception types:
with suppress(FileNotFoundError, PermissionError):
    os.remove(temp_file)
```

Exception groups (Python 3.11+)

```
# New in Python 3.11: ExceptionGroup for handling multiple simultaneous errors
# Useful in async/concurrent code where multiple tasks may fail at once

try:
    raise ExceptionGroup("multiple errors", [
        ValueError("bad value"),
        TypeError("bad type"),
        KeyError("bad key")
    ])
```

```

    })
except* ValueError as eg:          # except* (star) – new syntax
    print(f"Handled ValueError: {eg.exceptions}")
except* (TypeError, KeyError) as eg:
    print(f"Handled Type/Key Errors: {eg.exceptions}")

# except* doesn't stop after first match –
# ALL matching except* blocks run (unlike regular except)
# Unmatched exceptions are re-raised as a new ExceptionGroup

```

Interview Cheat Sheet

BaseException	→ root of ALL exceptions (rarely catch this)
Exception	→ root of all NORMAL exceptions (catch this)
SystemExit/KeyboardInterrupt	→ inherit from BaseException, not Exception
try block cost	→ near zero (CPython zero-cost exceptions, 3.11+)
raise cost	→ expensive: traceback capture, stack walk, frame all oc
Propagation order:	raise → search current frame → unwind → search caller
Block execution:	try → (except if error) → (else if no error) → finally
__context__	→ implicit chain (exception during exception handling)
__cause__	→ explicit chain (raise X from Y)
__suppress_context__	→ True when using raise X from Y or raise X from None
raise	→ re-raises current exception, preserves traceback
raise ExcType()	→ new exception, new traceback
raise X from Y	→ explicit chain, shows "direct cause"
raise X from None	→ suppresses context, hides implementation details
EAFP	→ Pythonic, fast when happy path is common
LBYL	→ better when errors are frequent, avoids exception cost
finally	→ ALWAYS runs (except os._exit())
finally return	→ OVERRIDES try/except return and SWALLOWS exceptions
else	→ runs only on NO exception, errors here don't go to except

```
Custom exceptions      → inherit from Exception, one base per app/package
                        → define __init__ with super().__init__(message)
                        → add rich attributes for programmatic handling
```

In Python, **attribute lookup** follows a strict resolution algorithm when you access something like:

```
obj.attr
```

Python must determine **where** `attr` exists. This process is governed by the **Attribute Lookup Order** and the **Method Resolution Order (MRO)**.

I'll first show the **full lookup flow**, then explain each step in detail.

1 Full Attribute Lookup Flow

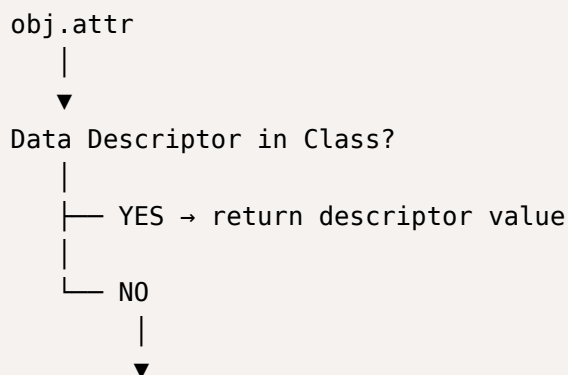
When you access:

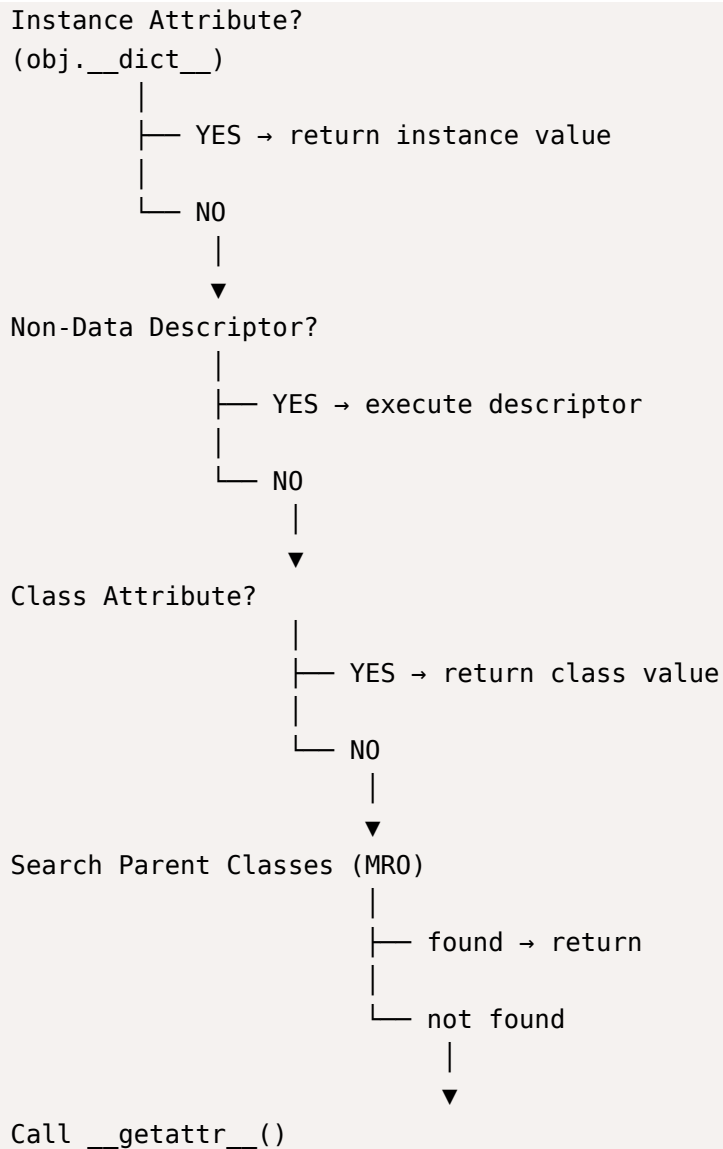
```
obj.attr
```

Python searches in this order:

1. Data Descriptor in the Class
2. Instance Dictionary (`obj.__dict__`)
3. Non-Data Descriptor in the Class
4. Class Dictionary (`Class.__dict__`)
5. Parent Classes via MRO
6. `__getattr__()` fallback

Visual flow:





If still not found:

`AttributeError`

2 Step 1 — Data Descriptors

Python **first checks data descriptors defined in the class**.

A **data descriptor** is any object that implements:

`__get__()` AND (`__set__()` or `__delete__()`)

Example: `@property`

Example:

```
class A:
    @property
    def x(self):
        return 100

obj = A()
print(obj.x)
```

Even if instance has , **descriptor wins**.

Why?

Because data descriptors have **highest priority**.

Visual:

```
Class
└─ x → property descriptor

Instance
└─ x → value

Result → property wins
```

Step 2 — Instance Dictionary

If no data descriptor exists, Python checks:

```
obj.__dict__
```

Example:

```
class A:
    pass

obj = A()
obj.x = 10

print(obj.x)
```

Memory:

```
obj.__dict__ = {  
    "x": 10  
}
```

Python returns:

```
10
```

4 Step 3 — Non-Data Descriptors

A **non-data descriptor** implements:

```
__get__()
```

but **not** `__set__()`.

Example: **methods**

```
class A:  
    def hello(self):  
        print("hi")
```

Internally:

```
A.__dict__["hello"] → function descriptor
```

When accessed:

```
obj.hello
```

Python converts it to:

```
bound method
```

Visualization:

```
function descriptor  
  |  
  ▼  
bind obj → method
```

5 Step 4 — Class Dictionary

If attribute isn't in the instance, Python checks:

```
Class.__dict__
```

Example:

```
class A:
    x = 50

obj = A()
print(obj.x)
```

Search:

```
obj.__dict__ → not found
A.__dict__ → found
```

Result:

```
50
```

6 Step 5 — Parent Classes (MRO)

If not in class, Python searches **parent classes using MRO**.

Example:

```
class A:
    x = 10

class B(A):
    pass

obj = B()
print(obj.x)
```

Search order:

```
B.__dict__
A.__dict__
object.__dict__
```

This order is defined by **MRO**.

You can see it using:


```
print(B.__mro__)
```

Output:

```
(B, A, object)
```

7 Step 6 — `__getattr__` Fallback

If attribute still isn't found, Python calls:

```
__getattr__(self, name)
```

Example:

```
class A:
    def __getattr__(self, name):
        return "Not Found"

obj = A()
print(obj.xyz)
```

Output:

```
Not Found
```

Why?

Because `xyz` was not found anywhere else.

8 Important Special Case — `__getattribute__`

Before all lookup happens, Python actually calls:

```
__getattribute__()
```

Example:

```
class A:
    def __getattribute__(self, name):
        print("Accessing:", name)
        return super().__getattribute__(name)
```

This method intercepts **all attribute access**.

Flow becomes:

```
obj.attr
|
▼
__getattr__()
|
▼
normal lookup
```

9 Complete Real Lookup Order

The **true complete lookup pipeline**:

```
obj.attr
|
▼
__getattr__()
|
▼
Data Descriptor
|
▼
Instance Dictionary
|
▼
Non-Data Descriptor
|
▼
Class Dictionary
|
▼
Parent Classes (MRO)
|
▼
__getattr__()
|
▼
AttributeError
```

10 Real Python Example Demonstrating Order

```
class Descriptor:
    def __get__(self, obj, owner):
        print("descriptor called")
        return 100

class A:
    x = Descriptor()

obj = A()
obj.x = 10

print(obj.x)
```

Output:

```
descriptor called
100
```

Why?

Because **data descriptors override instance attributes**.

1 1 Interview One-Line Answer

Python attribute lookup follows this order: `__getattribute__` → data descriptors → instance dictionary → non-data descriptors → class dictionary → parent classes via MRO → `__getattr__`.

If you'd like, I can also show you a **visual memory diagram of descriptors vs instance attributes**, which makes this concept **extremely easy to remember for interviews**.

Python Data Types — Complete Internals, Memory Management & Traps

1. Python's Type System Foundation

Everything is an object

```

# In Python, EVERY value is an object with:
# - A type      (what it is)
# - An identity (where it lives in memory)
# - A value     (what it holds)

x = 42
print(type(x))      # <class 'int'>
print(id(x))        # memory address (e.g., 140234567)
print(x)            # 42

# Even types are objects
print(type(int))     # <class 'type'>
print(type(type))    # <class 'type'> ← type is its own type

# Functions are objects
def foo(): pass
print(type(foo))     # <class 'function'>
print(id(foo))       # has a memory address

```

PyObject — the internal C structure

```

/* Every Python object in CPython is a PyObject at minimum */
typedef struct _object {
    Py_ssize_t ob_refcnt;    // reference count – for garbage collection
    PyTypeObject *ob_type;   // pointer to type object
} PyObject;

/* Integers also store the value */
typedef struct {
    PyObject ob_base;
    Py_ssize_t ob_ival;     // the actual int value
} PyLongObject;

```

```

import sys, ctypes

x = 42
print(sys.getsizeof(x))      # 28 bytes (int object overhead)
print(sys.getsizeof(True))   # 28 bytes (bool is subclass of int)
print(sys.getsizeof("hello")) # 54 bytes (str has per-char overhead)
print(sys.getsizeof([]))     # 56 bytes (empty list)
print(sys.getsizeof([1,2,3])) # 88 bytes (list + 3 pointers)

```

Reference counting — the core GC mechanism

```
import sys

a = [1, 2, 3]
print(sys.getrefcount(a))    # 2 (a + getrefcount's own arg)

b = a                        # ref count → 3
c = a                        # ref count → 4
del b                        # ref count → 3
del c                        # ref count → 2

# When ref count hits 0 → object is immediately deallocated
# (in CPython – other implementations differ)

# getrefcount always shows +1 because the function call itself
# creates a temporary reference to the argument
```

2. Integers (`int`)

Arbitrary precision

```
# Python ints have NO overflow – they grow as needed
big = 2 ** 1000
print(big)                # enormous number
print(type(big))           # <class 'int'> – still just int
print(sys.getsizeof(big))   # 160 bytes – much bigger than small int

# Size grows with value:
print(sys.getsizeof(0))     # 24 bytes
print(sys.getsizeof(2**30)) # 28 bytes
print(sys.getsizeof(2**60)) # 32 bytes
print(sys.getsizeof(2**90)) # 36 bytes
# Each additional 30 bits needs 4 more bytes
```

Integer interning — the BIGGEST trap

```
# CPython interns (caches) small integers: -5 to 256
# These are PRE-ALLOCATED at interpreter startup
# All references to these values point to the SAME object

a = 100
b = 100
```

```

print(a is b)      # True ← same object (interned)
print(id(a) == id(b))  # True

a = 257
b = 257
print(a is b)      # False ← different objects (NOT interned)
print(id(a) == id(b))  # False

# △ TRAP: this can be True in some contexts even for 257!
a = b = 257
print(a is b)      # True! ← same assignment, compiler optimization

# In interactive shell vs script – different behavior!
# The compiler optimizes same-value literals in the same code block

```

Interning edge cases

```

# Negative numbers: only -5 to -1 are interned
a = -5; b = -5; print(a is b)  # True
a = -6; b = -6; print(a is b)  # False

# Runtime computed values – not interned even in range
x = 100 + 0
y = 100
print(x is y)  # True (100 is in cache, so both point to cached obj)

x = 200 + 57    # = 257
y = 257
print(x is y)   # False – runtime computation, not interned

# RULE: NEVER use `is` to compare integers – ALWAYS use `==`

```

Integer memory layout

```

# Small ints (-5 to 256) are singletons
print(id(1) == id(1))      # True – same object every time

# Large ints are new objects each time (in expressions)
print(id(1000) == id(1000)) # Usually False in different expressions

# But same literal in same expression can be same object
x = 1000
print(id(x) == id(x))      # True – same variable, same object

```

Scenario — integer traps in real code

```
# ❌ WRONG — using `is` for integer comparison
def process(status_code):
    if status_code is 200:          # SyntaxWarning in Python 3.8+
        return "OK"                # works for 200 (interned) but WRONG approach

# ✅ CORRECT
def process(status_code):
    if status_code == 200:          # always use ==
        return "OK"

# ❌ TRAP — assuming int equality works with is
cache = {}
def get_cached(n):
    if n not in cache:
        cache[n] = n * 2
    return cache[n]

a = get_cached(300)
b = get_cached(300)
print(a is b)    # True (same object in dict) — accidental, don't rely on it
print(a == b)    # True — this is what you should check
```

3. Floats (**float**)

IEEE 754 double precision

```
# Python float = C double = 64-bit IEEE 754
# 1 sign bit, 11 exponent bits, 52 mantissa bits
# Range: ~5e-324 to ~1.8e+308
# Precision: ~15-17 significant decimal digits

import sys
print(sys.getsizeof(3.14))    # 24 bytes — always fixed size (unlike int)
print(sys.float_info)
# sys.float_info(max=1.7976931348623157e+308, ..., epsilon=2.220446049250313e-16)
```

The floating point trap — most famous Python gotcha

```
print(0.1 + 0.2)              # 0.30000000000000004
print(0.1 + 0.2 == 0.3)       # False ← !!
```

```
# Why? 0.1 cannot be represented exactly in binary:
# 0.1 in binary = 0.0001100110011001100... (repeating)
# Stored as closest 64-bit approximation

import decimal
print(decimal.Decimal(0.1))
# Decimal('0.1000000000000000055511151231257827021181583404541015625')
# This is what 0.1 ACTUALLY is in memory!
```

Correct float comparison

```
# ❌ WRONG
if 0.1 + 0.2 == 0.3:
    print("equal")      # never prints!

# ✅ Method 1: math.isclose (best for general use)
import math
print(math.isclose(0.1 + 0.2, 0.3))      # True
print(math.isclose(0.1 + 0.2, 0.3,
    rel_tol=1e-9, abs_tol=1e-12))      # True

# ✅ Method 2: round for display/comparison
print(round(0.1 + 0.2, 10) == round(0.3, 10))      # True

# ✅ Method 3: decimal module for exact arithmetic
from decimal import Decimal
print(Decimal('0.1') + Decimal('0.2') == Decimal('0.3'))      # True
# Note: Decimal('0.1') not Decimal(0.1) – the string form is exact
```

Float special values

```
import math

# Infinity
pos_inf = float('inf')
neg_inf = float('-inf')
print(pos_inf > 10**1000)      # True
print(neg_inf < -10**1000)      # True
print(math.isinf(pos_inf))      # True

# NaN – Not a Number
nan = float('nan')
print(nan == nan)      # False ← NaN is not equal to ITSELF!
```



```

print(nan != nan)          # True
print(math.isnan(nan))    # True ← only safe way to check NaN

# △ TRAP: NaN comparisons
values = [1.0, float('nan'), 3.0]
print(max(values))        # 3.0 or nan – undefined behavior!
print(sorted(values))     # unpredictable order due to NaN

# △ TRAP: float arithmetic overflow
print(1.8e308 * 2)        # inf – no exception, just inf!
print(1e-400)             # 0.0 – underflow to zero silently

```

Float interning

```

# Floats are NOT interned (except some implementations may cache 0.0)
a = 3.14
b = 3.14
print(a is b)             # False (usually) – different objects
print(a == b)             # True – same value

# But in same code block, compiler may optimize:
print(3.14 is 3.14)      # True in CPython (same literal, same object in code)
# Still: NEVER use `is` for floats

```

4. Booleans (**bool**)

bool is a subclass of int

```

print(issubclass(bool, int)) # True
print(isinstance(True, int)) # True
print(isinstance(True, bool)) # True

# True == 1, False == 0
print(True == 1)           # True
print(False == 0)          # True
print(True + True)         # 2 ← bool arithmetic!
print(True * 5)            # 5
print(False * 100)         # 0

# True and False are singletons
print(id(True) == id(True)) # True – always same object
print(True is True)         # True – safe to use `is` for booleans

```

Boolean traps

```
# ⚠ TRAP: bool in arithmetic
scores = [True, False, True, True] # represents pass/fail
print(sum(scores))    # 3 – counts True values (True==1)
# This is actually useful! But can be confusing

# ⚠ TRAP: is vs == with booleans
x = 1
print(x == True)      # True  (1 == 1)
print(x is True)      # False (1 is not the True singleton)

# ⚠ TRAP: bool subclass breaks type checking
def process(flag):
    if type(flag) == bool:      # True only for actual bool
        print("bool")
    if isinstance(flag, bool):  # True for bool
        print("isinstance bool")
    if isinstance(flag, int):    # Also True for bool!
        print("isinstance int")

process(True) # prints all three!

# ✅ Strict bool check:
def strict_bool(flag):
    if type(flag) is bool:     # use `is` for type comparison
        print("strictly bool")
```

Truthiness — what evaluates to False

```
# ALL of these are falsy:
falsy_values = [
    False,      # bool
    0,          # int zero
    0.0,        # float zero
    0j,         # complex zero
    "",         # empty string
    b"",        # empty bytes
    [],         # empty list
    (),         # empty tuple
    {},         # empty dict
    set(),      # empty set
    None,       # None
    # Any object where __bool__ returns False
```

```

    # or __len__ returns 0
]

# △ TRAP: empty vs None confusion
def get_items():
    return [] # empty list

items = get_items()
if not items:
    print("no items or None!") # triggers for BOTH [] and None!

# ✓ Be explicit when it matters:
if items is None:
    print("function failed")
elif len(items) == 0:
    print("empty result")

```

5. Strings (`str`)

String internals — immutable sequence of Unicode code points

```

# Python 3 strings are Unicode by default
# Internal representation depends on the highest code point:
# - Latin-1 (1 byte/char) for code points <= 255
# - UCS-2 (2 bytes/char) for code points <= 65535
# - UCS-4 (4 bytes/char) for code points > 65535

import sys
print(sys.getsizeof("")) # 49 bytes (empty string overhead)
print(sys.getsizeof("a")) # 50 bytes (+1 byte for latin-1 char)
print(sys.getsizeof("hello")) # 54 bytes (+5 bytes)
print(sys.getsizeof("α")) # 76 bytes (UCS-2, 2 bytes/char)
print(sys.getsizeof("ℹ")) # 80 bytes (UCS-4, 4 bytes/char)

```

String interning

```

# Python automatically interns strings that look like identifiers
# (only letters, digits, underscores)

a = "hello"
b = "hello"
print(a is b) # True – interned (looks like identifier)

```

```

a = "hello world"      # has a space – NOT auto-interned
b = "hello world"
print(a is b)          # False (usually) – not interned

# Force interning with sys.intern()
import sys
a = sys.intern("hello world")
b = sys.intern("hello world")
print(a is b)          # True – explicitly interned

# △ TRAP: relying on string interning
a = "test"
b = "te" + "st"        # compile-time constant folding → interned
print(a is b)          # True (compiler folds the concatenation)

a = "test"
b = "te"
c = b + "st"           # runtime concatenation – NOT interned
print(a is c)          # False – don't rely on this!

```

String immutability and memory

```

# Strings are immutable – every "modification" creates a new object

s = "hello"
print(id(s))           # some address

s = s + " world"       # creates a NEW string object
print(id(s))           # different address – old "hello" may be GC'd

# △ TRAP: String concatenation in a loop –  $O(n^2)$  memory!
# Each += creates a new string, copying all previous content

# ✗ SLOW –  $O(n^2)$  time and memory
result = ""
for i in range(10000):
    result += str(i)    # creates 10000 intermediate string objects!

# ✓ FAST –  $O(n)$  – use list + join
parts = []
for i in range(10000):
    parts.append(str(i))
result = "".join(parts) # single allocation, single copy

```

```
# ✅ Also fast – list comprehension + join
result = "".join(str(i) for i in range(10000))
```

String methods — traps

```
# ⚠ TRAP: split() vs split(' ')
s = "  hello  world  "
print(s.split())          # ['hello', 'world'] – strips and splits on any white
                           space
print(s.split(' '))       # ['', '', 'hello', '', '', 'world', '', ''] – exact
                           split!

# ⚠ TRAP: strip() only removes from ends
s = "  hello  world  "
print(s.strip())          # "hello  world" – middle spaces preserved!

# ⚠ TRAP: replace replaces ALL occurrences
s = "aabbbaa"
print(s.replace("a", "x")) # "xxbbxx" – ALL a's replaced
print(s.replace("a", "x", 2)) # "xxbbbaa" – only first 2

# ⚠ TRAP: in checks for substring, not just characters
print("el" in "hello")    # True – substring check
print("lo" in "hello")    # True
print("ole" in "hello")   # False

# ⚠ TRAP: string multiplication
s = "-" * 0              # "" – empty string
s = "-" * -1             # "" – negative multiplier gives empty string, not error

# ⚠ TRAP: index vs find
s = "hello"
# s.index("z")            # ValueError – raises if not found
s.find("z")               # -1 – returns -1 if not found (no exception)
```

String formatting internals

```
# f-strings are FASTEST – compiled to bytecode at parse time
name = "Alice"
age = 30

# f-string – evaluated at runtime, fastest
s1 = f"Name: {name}, Age: {age}"
```

```

# .format() – slightly slower, more flexible
s2 = "Name: {}, Age: {}".format(name, age)

# % formatting – old style, slowest for complex cases
s3 = "Name: %s, Age: %d" % (name, age)

# ⚠ TRAP: f-string with quotes
# d = {"key": "value"}
# print(f"Value: {d["key"]}") # SyntaxError in Python < 3.12
print(f"Value: {d['key']}") # use different quotes inside
# Python 3.12+ allows same quotes inside f-strings

# ⚠ TRAP: f-string side effects
counter = 0
def increment():
    global counter
    counter += 1
    return counter

s = f"{increment()} {increment()}" # side effects in f-strings!
print(s) # "1 2"
print(counter) # 2 – function called twice

```

6. Lists (**List**)

List internals — dynamic array

```

# Python list is a dynamic array (like ArrayList in Java)
# Internally: array of POINTERS to PyObjects
# Not stored contiguously by value – pointers are contiguous

import sys
print(sys.getsizeof([])) # 56 bytes (empty list)
print(sys.getsizeof([1])) # 64 bytes (+8 bytes for one pointer)
print(sys.getsizeof([1, 2])) # 72 bytes
print(sys.getsizeof([1, 2, 3])) # 80 bytes
# Each element = 8 bytes (64-bit pointer)

```

Over-allocation — how list growth works

```

import sys

lst = []

```

```

prev_size = sys.getsizeof(lst)
print(f"size=0: {prev_size} bytes, capacity inferred")

for i in range(20):
    lst.append(i)
    new_size = sys.getsizeof(lst)
    if new_size != prev_size:
        print(f"size={len(lst)}: {new_size} bytes – REALLOCATION")
        prev_size = new_size

# Output shows reallocation at: 1, 5, 9, 17, 25...
# Growth pattern: 0→4→8→16→25→35→46... (roughly 1.125x growth)
# Each reallocation: allocate new array, copy all pointers

```

List memory — shallow container

```

# Lists store POINTERS, not values
# All elements are references to objects elsewhere in memory

a = [1, 2, 3]
b = [1, 2, 3]
# The integers 1,2,3 may be the SAME objects (interned)
# But the list objects (a, b) are different

print(a is b)      # False – different list objects
print(a == b)      # True  – same contents

# Shallow copy trap:
import copy
original = [[1, 2], [3, 4]]
shallow = original.copy()      # or list(original) or original[:]

shallow[0].append(99)          # modifies the INNER list
print(original)  # [[1, 2, 99], [3, 4]] ← original CHANGED!

# Fix: deep copy
deep = copy.deepcopy(original)
deep[0].append(99)
print(original)  # [[1, 2, 99], [3, 4]] ← unchanged

```

List traps — all scenarios

```

# △ TRAP 1: Mutable default argument
def add_item(item, lst=[]):      # lst created ONCE, shared across calls!

```

```

    lst.append(item)
    return lst

print(add_item(1))    # [1]
print(add_item(2))    # [1, 2] ← 2 is in same list!
print(add_item(3))    # [1, 2, 3]

# ✅ Fix:
def add_item(item, lst=None):
    if lst is None:
        lst = []
    lst.append(item)
    return lst

# ⚠ TRAP 2: List multiplication with mutable objects
matrix = [[0] * 3] * 3          # looks like 3x3 matrix
matrix[0][0] = 1
print(matrix)    # [[1, 0, 0], [1, 0, 0], [1, 0, 0]] ← ALL rows changed!
# All 3 rows are the SAME list object

# ✅ Fix:
matrix = [[0] * 3 for _ in range(3)]    # each row is a new list
matrix[0][0] = 1
print(matrix)    # [[1, 0, 0], [0, 0, 0], [0, 0, 0]] ✓

# ⚠ TRAP 3: Modifying list while iterating
lst = [1, 2, 3, 4, 5]
for item in lst:
    if item % 2 == 0:
        lst.remove(item)    # modifies list during iteration!
print(lst)    # [1, 3, 5]? NO! → [1, 3, 5] sometimes, but skips elements

# To see the bug clearly:
lst = [1, 2, 3, 4, 5, 6]
for item in lst:
    if item % 2 == 0:
        lst.remove(item)
print(lst)    # [1, 3, 5] – looks right but actually skipped some checks!

# ✅ Fix 1: iterate over a copy
for item in lst[:]:
    if item % 2 == 0:
        lst.remove(item)

# ✅ Fix 2: list comprehension

```



```

lst = [item for item in lst if item % 2 != 0]

# △ TRAP 4: extend vs append with list
lst = [1, 2, 3]
lst.append([4, 5])    # [1, 2, 3, [4, 5]] ← nested list!
lst.extend([4, 5])    # [1, 2, 3, 4, 5]    ← flattened

# △ TRAP 5: negative indexing past bounds
lst = [1, 2, 3]
print(lst[-1])        # 3 ← fine
# print(lst[-4])      # IndexError – no silent wrap-around past length

# △ TRAP 6: sort() vs sorted()
lst = [3, 1, 2]
new = lst.sort()       # returns None! sorts IN PLACE
print(new)             # None ← trap!
new = sorted(lst)      # returns new sorted list
print(new)             # [1, 2, 3]

```

List performance characteristics

```

# Time complexities:
# append(x)          0(1) amortized – 0(n) on reallocation
# pop()              0(1) – from end
# pop(0)             0(n) – from front! shifts all elements
# insert(0, x)       0(n) – shifts all elements
# remove(x)          0(n) – linear search + shift
# x in list           0(n) – linear search
# list[i]            0(1) – random access
# len(list)          0(1) – stored as field

# △ TRAP: using list as queue (pop from front)
from collections import deque
queue_list = [1, 2, 3, 4, 5]
queue_list.pop(0)     # 0(n) – bad for large lists!

# ✅ Use deque for queue operations
queue = deque([1, 2, 3, 4, 5])
queue.popleft()       # 0(1) – deque is a doubly-linked list

```

7. Tuples (**tuple**)

Tuple internals — immutable sequence

```

import sys

# Tuples are more memory-efficient than lists
print(sys.getsizeof(()))      # 40 bytes (empty tuple)
print(sys.getsizeof((1,)))    # 48 bytes
print(sys.getsizeof((1, 2, 3))) # 64 bytes

print(sys.getsizeof([]))      # 56 bytes (empty list)
print(sys.getsizeof([1, 2, 3])) # 80 bytes

# Tuple is 16+ bytes smaller – no over-allocation, no growth metadata

```

Tuple interning and caching

```

# CPython caches empty tuple – always same object
a = ()
b = ()
print(a is b)    # True – singleton

# Small tuples of interned objects may be cached
a = (1, 2)
b = (1, 2)
print(a is b)    # True (CPython optimizes this – implementation detail)

# Larger or complex tuples are NOT cached
a = (1, 2, 3, 4, 5)
b = (1, 2, 3, 4, 5)
print(a is b)    # False (usually)

```

Tuple traps

```

# ⚠ TRAP 1: Single-element tuple needs trailing comma
t = (1)      # NOT a tuple – just parenthesized int!
print(type(t)) # <class 'int'>

t = (1,)     # This IS a tuple
print(type(t)) # <class 'tuple'>

t = 1,       # Also a tuple (parentheses optional)
print(type(t)) # <class 'tuple'>

# ⚠ TRAP 2: Tuple immutability is SHALLOW
t = ([1, 2], [3, 4])

```

```

t[0].append(99)          # tuple itself unchanged, but inner list IS mutable!
print(t)    # ([1, 2, 99], [3, 4]) ← inner list modified

# t[0] = [1,2,99]        # TypeError: tuple object does not support item assignment
# But t[0].append() works because we're not changing the tuple's reference

# △ TRAP 3: Tuple hashability – same shallow trap
# t = ([1,2], 3)
# hash(t)    # TypeError: unhashable type: 'list'
# Tuple is only hashable if ALL elements are hashable

t = (1, 2, 3)
print(hash(t))    # works – all ints

# △ TRAP 4: tuple() on string
t = tuple("hello")
print(t)    # ('h', 'e', 'l', 'l', 'o') ← not ('hello',)!

```

Named tuples — structured tuples

```

from collections import namedtuple
from typing import NamedTuple

# Classic style
Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
print(p.x, p.y)          # 3 4
print(p[0], p[1])        # 3 4 – still indexable
print(p._asdict())        # {'x': 3, 'y': 4}

# Modern style with types
class Point3D(NamedTuple):
    x: float
    y: float
    z: float = 0.0    # default value

p = Point3D(1.0, 2.0)
print(p)                # Point3D(x=1.0, y=2.0, z=0.0)
print(p._replace(x=10.0)) # Point3D(x=10.0, y=2.0, z=0.0)

# Memory: namedtuple is still a tuple underneath
print(sys.getsizeof(p))  # same as equivalent tuple

```

8. Dictionaries (`dict`)

Dict internals — hash table

```
# Python dict uses an open-addressing hash table
# Since Python 3.6: insertion ORDER is preserved (CPython)
# Since Python 3.7: order preservation is GUARANTEED by spec

# Internal structure (simplified):
# - indices array: sparse array mapping hash → entry index
# - entries array: dense array of (hash, key, value) triplets

import sys
print(sys.getsizeof({}))          # 64 bytes (empty)
print(sys.getsizeof({'a': 1}))    # 232 bytes (first entry triggers allo
c)
print(sys.getsizeof({'a':1,'b':2})) # 232 bytes (same initial allocation)
```

How hashing works internally

```
# dict[key] lookup:
# 1. Compute hash(key)
# 2. hash % table_size → slot index
# 3. Check if slot's key == our key
# 4. If collision → probe next slot (open addressing)
# 5. If found → return value; if empty slot → KeyError

# Hash examples:
print(hash("hello"))             # some large integer (varies per run – hash randomi
zation)
print(hash(42))                  # 42 ← int hashes to themselves (small ints)
print(hash(3.14))                # consistent with float value
print(hash((1, 2)))              # tuple hash based on contents

# Hash randomization (Python 3.3+):
# PYTHONHASHSEED=0 python → disables randomization (reproducible hashes)
# Default: random seed per interpreter start
# Prevents hash-collision DoS attacks
```

Dict key requirements

```
# Keys MUST be hashable (immutable and implement __hash__)
d = {}
```

```

d[1] = "int key"           # ✓
d["hello"] = "str key"    # ✓
d[(1, 2)] = "tuple key"   # ✓
d[3.14] = "float key"     # ✓

# d[[1, 2]] = "list key"   # TypeError: unhashable type: 'list'
# d[{1, 2}] = "set key"    # TypeError: unhashable type: 'set'
# d[{'a':1}] = "dict key"  # TypeError: unhashable type: 'dict'

# △ TRAP: float keys and equality
d = {}
d[1] = "one"
print(d[1.0])    # "one" ← 1 == 1.0 and hash(1) == hash(1.0)!

d[1.0] = "one-float"
print(d)         # {1: 'one-float'} ← key stays as int 1, value updated!
# Because 1 == 1.0, they are considered the SAME key

```

Dict operations and traps

```

# △ TRAP 1: KeyError vs .get()
d = {'a': 1}
# d['b']           # KeyError!
d.get('b')         # None – safe
d.get('b', 0)      # 0 – with default

# △ TRAP 2: Modifying dict while iterating
d = {'a': 1, 'b': 2, 'c': 3}
# for k in d:
#     if k == 'b':
#         del d[k]  # RuntimeError: dictionary changed size during iteration

# ✅ Fix: iterate over copy of keys
for k in list(d.keys()):
    if k == 'b':
        del d[k]

# △ TRAP 3: dict.update() overwrites silently
d1 = {'a': 1, 'b': 2}
d2 = {'b': 99, 'c': 3}
d1.update(d2)
print(d1)         # {'a': 1, 'b': 99, 'c': 3} ← b was silently overwritten!

# △ TRAP 4: Merging dicts – which wins?

```

```

d1 = {'a': 1}
d2 = {'a': 2}
merged = {**d1, **d2}    # d2 wins – rightmost wins
print(merged)    # {'a': 2}

merged = d1 | d2        # Python 3.9+ merge operator, d2 wins
print(merged)    # {'a': 2}

d1 |= d2              # Python 3.9+ update in place
print(d1)           # {'a': 2}

# △ TRAP 5: dict views are LIVE
d = {'a': 1, 'b': 2}
keys = d.keys()        # live view, not a copy!
d['c'] = 3
print(keys)            # dict_keys(['a', 'b', 'c']) ← c appeared!
print(list(keys))      # freeze to list if you need a snapshot

# △ TRAP 6: Shallow copy
d = {'a': [1, 2, 3]}
copy = d.copy()
copy['a'].append(99)
print(d)               # {'a': [1, 2, 99]} ← original mutated!

```

defaultdict and Counter

```

from collections import defaultdict, Counter

# defaultdict – auto-creates missing keys
dd = defaultdict(list)
dd['a'].append(1)    # no KeyError – creates [] automatically
dd['a'].append(2)
print(dd)    # defaultdict(<class 'list'>, {'a': [1, 2]})

# △ TRAP: defaultdict creates key on ACCESS (not just assignment)
dd = defaultdict(int)
if dd['missing']:    # ← this CREATES the key 'missing' with value 0!
    print("truthy")
print('missing' in dd)    # True – it was created!

# Counter – specialized dict for counting
c = Counter("hello world")
print(c)                # Counter({'l': 3, 'o': 2, 'h': 1, ...})
print(c['l'])            # 3

```

```

print(c['z'])      # 0 ← returns 0 for missing, not KeyError

c1 = Counter("hello")
c2 = Counter("world")
print(c1 + c2)     # combined counts
print(c1 - c2)     # subtract counts (drops zeros and negatives)
print(c1.most_common(2))  # [('l', 2), ('h', 1)]

```

Dict performance

```

# Average time complexities:
# d[key]          0(1) average – 0(n) worst case (hash collisions)
# d[key] = val    0(1) average
# del d[key]      0(1) average
# key in d        0(1) average
# len(d)          0(1)
# d.keys()        0(1) – returns view
# list(d.keys())  0(n) – materializes the view

# Dict vs list for membership testing:
import timeit
lst = list(range(10000))
st = set(range(10000))
dct = dict.fromkeys(range(10000))

t1 = timeit.timeit(lambda: 9999 in lst, number=100000)
t2 = timeit.timeit(lambda: 9999 in st, number=100000)
t3 = timeit.timeit(lambda: 9999 in dct, number=100000)
print(f"list: {t1:.3f}s")  # ~0.5s
print(f"set: {t2:.3f}s")  # ~0.003s
print(f"dict: {t3:.3f}s")  # ~0.003s

```

9. Sets (**set** and **frozenset**)

Set internals — hash table without values

```

# Set is essentially a dict with only keys, no values
# Same hash table mechanics – 0(1) membership testing

import sys
print(sys.getsizeof(set()))      # 216 bytes
print(sys.getsizeof({1}))        # 216 bytes

```

```
print(sys.getsizeof({1,2,3,4,5})) # 216 bytes (initial table size)
print(sys.getsizeof(set(range(9)))) # 728 bytes (table grew)
```

Set operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)    # {1,2,3,4,5,6} – union
print(a & b)    # {3,4}         – intersection
print(a - b)    # {1,2}         – difference (in a but not b)
print(a ^ b)    # {1,2,5,6}     – symmetric difference (in one but not both)

print(a.issubset({1,2,3,4,5})) # True
print(a.issuperset({1,2}))     # True
print(a.isdisjoint({7,8,9}))   # True (no common elements)
```

Set traps

```
# ⚠ TRAP 1: Empty set literal
s = {}          # This is a dict, NOT a set!
print(type(s))  # <class 'dict'>

s = set()       # This is an empty set
print(type(s))  # <class 'set'>

# ⚠ TRAP 2: Sets are UNORDERED – no indexing
s = {3, 1, 2}
# s[0] # TypeError: 'set' object is not subscriptable
# BUT: iteration order is consistent within one run (not guaranteed across runs)

# ⚠ TRAP 3: Set mutation during iteration
s = {1, 2, 3, 4, 5}
# for x in s:
#     if x % 2 == 0:
#         s.remove(x) # RuntimeError: Set changed size during iteration

# ✅ Fix:
s = {x for x in s if x % 2 != 0} # set comprehension

# ⚠ TRAP 4: frozenset is needed for nested sets
# s = {{1,2}, {3,4}} # TypeError: unhashable type: 'set'
s = {frozenset({1,2}), frozenset({3,4})} # ✓ frozensets are hashable
```



```
# Δ TRAP 5: Adding mutable objects
s = set()
lst = [1, 2, 3]
# s.add(lst)      # TypeError: unhashable type: 'list'
s.add(tuple(lst))  # ✓ convert to hashable first

# Δ TRAP 6: Set of floats
s = {0.1 + 0.2, 0.3}
print(len(s))     # 2! They're different values despite math saying equal
print(0.1 + 0.2 == 0.3)  # False – same float precision issue
```

10. `None` , `type(None)` , and `NoneType`

```
# None is a singleton – only ONE None object exists
a = None
b = None
print(a is b)      # True – ALWAYS use `is` to check for None
print(a is None)   # True ← correct way
print(a == None)   # True ← works but wrong style (can be overridden by __eq__)

# Δ TRAP: == None can lie
class Weird:
    def __eq__(self, other): return True  # always returns True!

w = Weird()
print(w == None)    # True ← wrong!
print(w is None)    # False ← correct

# None's type
print(type(None))    # <class 'NoneType'>
print(isinstance(None, type(None)))  # True

# Δ TRAP: None as default vs missing
def func(value=None):
    if value is None:
        print("no value provided OR None was explicitly passed")
    # Can't distinguish between func() and func(None)!

# ✓ Use a sentinel for "truly not provided":
_MISSING = object()  # unique sentinel object
```

```
def func(value=_MISSING):
    if value is _MISSING:
        print("no value provided")
    elif value is None:
        print("None explicitly passed")
    else:
        print(f"value is {value}")
```

11. bytes , bytearray , memoryview

```
# bytes – immutable sequence of integers 0-255
b = b"hello"
print(type(b))          # <class 'bytes'>
print(b[0])             # 104 (ASCII for 'h') – indexing gives int, not byte!
print(b[0:1])           # b'h' – slicing gives bytes
# b[0] = 105            # TypeError: bytes is immutable

# bytearray – mutable version
ba = bytearray(b"hello")
ba[0] = 72               # 'H'
print(ba)               # bytearray(b'Hello')

# ⚠ TRAP: bytes vs str
s = "hello"
b = b"hello"
print(s == b)           # False – never equal, different types
# print(s + b)          # TypeError: can't concat str to bytes

# memoryview – zero-copy view into buffer
data = bytearray(range(10))
view = memoryview(data)
print(bytes(view[2:5]))  # b'\x02\x03\x04' – slice without copy!

# Modifying through memoryview modifies original
view[0] = 99
print(data[0])          # 99
```

12. Type Conversion Traps

```
# int() conversions
print(int("42"))        # 42
print(int("42.5"))       # ValueError! (can't convert decimal string directly)
```

```

print(int(42.9))      # 42 – TRUNCATES, not rounds!
print(int(-42.9))     # -42 – truncates toward zero
print(int(True))      # 1
print(int(False))     # 0
print(int("0xFF", 16)) # 255 – base conversion
print(int("0b101", 2)) # 5 – binary

# float() conversions
print(float("inf"))   # inf
print(float("nan"))   # nan
print(float("1e10"))  # 10000000000.0

# str() vs repr()
print(str("hello"))   # hello (no quotes)
print(repr("hello"))  # 'hello' (with quotes)
print(str(None))      # 'None' (string)
print(repr(None))     # 'None'

# bool() – what evaluates to True
print(bool(0))        # False
print(bool(0.0))      # False
print(bool(""))       # False
print(bool("False"))  # True! ← non-empty string is truthy
print(bool([]))       # False
print(bool([False]))  # True! ← non-empty list is truthy
print(bool(None))     # False

# list() vs [x]
print(list("abc"))    # ['a', 'b', 'c'] – iterates string
print(["abc"])        # ['abc'] – wraps in list
print(list((1,2,3)))  # [1, 2, 3] – converts tuple

```

Interview Scenario Questions & Answers

Scenario 1 — Output prediction

```

a = [1, 2, 3]
b = a
b += [4, 5]      # ← list += uses __iadd__ which calls extend IN PLACE
print(a)         # [1, 2, 3, 4, 5] ← a and b are same object!

a = (1, 2, 3)
b = a

```

```

b += (4, 5)      # ← tuple += creates NEW tuple (immutable)
print(a)        # (1, 2, 3) ← a unchanged
print(b)        # (1, 2, 3, 4, 5) ← b is new object
print(a is b)   # False

```

Scenario 2 — Dict/set gotcha

```

d = {}
d[True] = "true"
d[1] = "one"
d[1.0] = "one-float"
print(d)      # {True: 'one-float'}
# True == 1 == 1.0 and hash(True)==hash(1)==hash(1.0)
# All three map to SAME slot – each overwrites value, key stays as True

```

Scenario 3 — Closure + mutable default

```

funcs = []
for i in range(3):
    def f(x=i):      # capture i as default arg (evaluated at definition time)
        return x
    funcs.append(f)

print([f() for f in funcs])  # [0, 1, 2] ← default arg captures work

funcs = []
for i in range(3):
    def f():
        return i      # closure – captures variable i, not value
    funcs.append(f)

print([f() for f in funcs])  # [2, 2, 2] ← all see final value of i

```

Scenario 4 — Memory and identity

```

a = 256
b = 256
print(a is b)  # True (interned)

a = 257
b = 257
print(a is b)  # False (not interned) – in separate statements!

```

```
# But in same expression:
print(257 is 257)    # True (compiler uses same constant object)
```

Scenario 5 — Unpacking traps

```
# △ Extended unpacking
a, *b, c = [1, 2, 3, 4, 5]
print(a)    # 1
print(b)    # [2, 3, 4] ← always a list, even if one element
print(c)    # 5

a, *b, c = [1, 2]
print(a)    # 1
print(b)    # [] ← empty list
print(c)    # 2

# △ Swap uses tuple packing/unpacking
a, b = 1, 2
a, b = b, a    # Python creates tuple (b, a) then unpacks
# No temp variable needed – fully atomic
```

Memory Management Summary

Object lifecycle:

- created → ref count = 1
- assigned/passed → ref count++
- deleted/out of scope → ref count--
- ref count == 0 → immediately deallocated (CPython)

Interning:

- int: -5 to 256 (always same object)
- str: identifier-like strings (auto), or `sys.intern()` (manual)
- float: NOT interned
- tuple: empty tuple is singleton; small tuples may be cached
- bool: True/False are singletons

Shallow vs Deep:

- = (assignment) → same object (ref copy)
- `.copy()` / `[:]` → shallow copy (new container, same elements)
- `copy.deepcopy()` → deep copy (recursive, new everything)

Mutable: list, dict, set, bytearray

```
Immutable: int, float, bool, str, bytes, tuple, frozenset, None
```

Golden rules:

```
NEVER use `is` for value comparison (only for None, True, False singletons)
ALWAYS use == for value equality
NEVER mutate while iterating (list, dict, set)
ALWAYS use deep copy for nested mutable structures
NEVER use mutable default arguments in functions
ALWAYS use `is None` to check for None (never == None)
```

4 Pillars of OOP in Python — Complete Internals, Memory & Traps

1. ENCAPSULATION

What it really means internally

```
# Encapsulation = bundling data + methods that operate on that data
# + controlling ACCESS to that data
# Python does NOT have true private members – it's convention + name mangling
```

```
class BankAccount:
    # Public    → accessible everywhere
    bank_name = "PyBank"

    # Protected → convention: single underscore (still accessible!)
    _interest_rate = 0.05

    # Private   → name mangling: double underscore
    __balance = 0      # stored as _BankAccount__balance internally

    def __init__(self, owner, balance):
        self.owner = owner      # public instance attr
        self._account_id = id(self) # protected
        self.__balance = balance # private – name mangled

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.__balance += amount    # controlled mutation

    def withdraw(self, amount):
```

```

        if amount > self.__balance:
            raise ValueError("Insufficient funds")
        self.__balance -= amount

    def get_balance(self):          # controlled READ access
        return self.__balance

acc = BankAccount("Alice", 1000)

```

Name mangling — the real mechanism

```

# Python transforms __name to _ClassName__name at COMPILE TIME
# This happens in the class body only – not outside

print(acc.__dict__)
# {'owner': 'Alice', '_account_id': ..., '_BankAccount__balance': 1000}
#                                     ^^^^^^^^^^^^^^^^^^^^^
#                                     stored with mangled name!

# Direct access FAILS:
# acc.__balance          # AttributeError: no attribute '__balance'

# But you CAN still access via mangled name (no true privacy!):
print(acc._BankAccount__balance)  # 1000 – Python's "we're all adults" policy

# ⚠ TRAP: name mangling in subclass
class SavingsAccount(BankAccount):
    def show_balance(self):
        # return self.__balance    # ← looks for _SavingsAccount__balance!
        return self._BankAccount__balance  # ← must use parent's mangled name

# ⚠ TRAP: double underscore in module scope – NO mangling
__version__ = "1.0"  # this is NOT mangled – mangling only inside class body

```

Properties — the Pythonic encapsulation tool

```

class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius    # protected backing attribute

    @property
    def celsius(self):
        """Getter – called on obj.celsius"""

```

```

        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Setter – called on obj.celsius = value"""
        if value < -273.15:
            raise ValueError(f"Temperature below absolute zero: {value}")
        self._celsius = value

    @celsius.deleter
    def celsius(self):
        """Deleter – called on del obj.celsius"""
        print("Resetting temperature")
        self._celsius = 0

    @property
    def fahrenheit(self):
        """Computed property – no setter (read-only)"""
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        self.celsius = (value - 32) * 5/9    # goes through celsius setter!

t = Temperature(25)
print(t.celsius)        # 25    ← calls getter
t.celsius = 100         # calls setter
print(t.fahrenheit)     # 212.0 ← computed property
t.fahrenheit = 32       # calls fahrenheit setter → celsius setter → -0.0
del t.celsius           # calls deleter

```

Property internals — descriptor protocol

```

# @property is implemented as a descriptor
# It implements __get__, __set__, __delete__

class property_manual:
    def __init__(self, fget=None, fset=None, fdel=None):
        self.fget = fget
        self.fset = fset
        self.fdel = fdel

    def __get__(self, obj, objtype=None):
        if obj is None:

```



```

        return self          # accessed from class, not instance
    if self.fget is None:
        raise AttributeError("unreadable attribute")
    return self.fget(obj)

    def __set__(self, obj, value):
        if self.fset is None:
            raise AttributeError("can't set attribute")
        self.fset(obj, value)

    def __delete__(self, obj):
        if self.fdel is None:
            raise AttributeError("can't delete attribute")
        self.fdel(obj)

    def setter(self, fset):
        return type(self)(self.fget, fset, self.fdel)

    def deleter(self, fdel):
        return type(self)(self.fget, self.fset, fdel)

```

__slots__ — memory optimization + encapsulation

```

import sys

class WithDict:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class WithSlots:
    __slots__ = ('x', 'y')    # replaces __dict__ with fixed-size array

    def __init__(self, x, y):
        self.x = x
        self.y = y

a = WithDict(1, 2)
b = WithSlots(1, 2)

print(sys.getsizeof(a))      # 48 bytes
print(sys.getsizeof(a.__dict__)) # 104 bytes → total ~152 bytes
print(sys.getsizeof(b))      # 56 bytes (no __dict__ at all!)

```

```

# △ __slots__ traps:
# b.z = 3          # AttributeError – can't add new attributes!
# b.__dict__       # AttributeError – no __dict__!

# △ TRAP: __slots__ in inheritance
class Base:
    __slots__ = ('x',)

class Child(Base):
    # If Child doesn't define __slots__, it gets __dict__ anyway!
    pass          # Child instances have BOTH slot x AND __dict__

class ProperChild(Base):
    __slots__ = ('y',)    # adds y slot, inherits x slot, no __dict__

# △ TRAP: __slots__ with multiple inheritance
class A:
    __slots__ = ('x',)

class B:
    __slots__ = ('y',)

class C(A, B):
    __slots__ = ()        # don't re-declare inherited slots!
    # if you declare __slots__ = ('x',) here – duplicate slot warning

# △ TRAP: pickling with __slots__
class Slotted:
    __slots__ = ('x', 'y')

    def __getstate__(self):    # must implement for pickling
        return {'x': self.x, 'y': self.y}

    def __setstate__(self, state):
        self.x = state['x']
        self.y = state['y']

```

Encapsulation scenarios

```

# SCENARIO 1: What prints?
class Counter:
    __count = 0    # class-level private

    def increment(self):

```

```

    Counter.__count += 1    # accesses via class name – works
    # self.__count += 1    # this would create INSTANCE attr!

    def get(self):
        return Counter.__count

c1 = Counter()
c2 = Counter()
c1.increment()
c2.increment()
print(c1.get())    # 2 ← shared class attribute

# SCENARIO 2: Property bypasses
class Person:
    def __init__(self, age):
        self.age = age    # ← calls setter!

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0: raise ValueError
        self._age = value

p = Person(-1)    # raises ValueError – __init__ goes through setter!
# Direct __dict__ bypass:
p.__dict__['age'] = -1    # bypasses setter – sets backing attr directly!
print(p.age)    # -1 – validation bypassed via __dict__!

# SCENARIO 3: Name mangling trap
class A:
    def __init__(self):
        self.__x = 10    # stored as _A__x

class B(A):
    def get_x(self):
        return self.__x    # looks for _B__x – AttributeError!

b = B()
# b.get_x()    # AttributeError: 'B' object has no attribute '_B__x'

```

2. ABSTRACTION

Abstract Base Classes — the mechanism

```
from abc import ABC, abstractmethod, abstractproperty
import inspect

class Shape(ABC):
    """Abstract class – cannot be instantiated directly"""

    @abstractmethod
    def area(self) -> float:
        """Every shape MUST implement area"""
        ... # or pass – body is ignored but documents the interface

    @abstractmethod
    def perimeter(self) -> float:
        """Every shape MUST implement perimeter"""
        ...

    @property
    @abstractmethod
    def name(self) -> str:
        """Abstract property – subclass must implement as property"""
        ...

    def describe(self):
        """Concrete method – shared implementation"""
        return f"{self.name}: area={self.area():.2f}, perimeter={self.perimeter():.2f}"

# Shape() # TypeError: Can't instantiate abstract class Shape
#         # with abstract methods: area, name, perimeter

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        import math
        return math.pi * self.radius ** 2

    def perimeter(self):
        import math
```

```

        return 2 * math.pi * self.radius

    @property
    def name(self):
        return "Circle"

c = Circle(5)
print(c.describe())    # Circle: area=78.54, perimeter=31.42

```

How ABC works internally

```

# ABCMeta tracks abstract methods via __abstractmethods__ frozenset

print(Shape.__abstractmethods__)    # frozenset({'area', 'perimeter', 'name'})
print(Circle.__abstractmethods__)    # frozenset() – empty, all implemented

# Python checks __abstractmethods__ at instantiation time
# in type.__call__ → if frozenset is non-empty → TypeError

# You can check if a class is abstract:
print(inspect.isabstract(Shape))     # True
print(inspect.isabstract(Circle))    # False

# △ TRAP: Partially implementing abstract class
class PartialShape(Shape):
    def area(self):
        return 0
    # perimeter and name not implemented!

# PartialShape()    # TypeError – still abstract!
print(PartialShape.__abstractmethods__)    # frozenset({'name', 'perimeter'})

```

__abstractmethods__ manipulation — the loophole

```

# △ LOOPHOLE: You can bypass abstract enforcement!
class BadConcrete(Shape):
    pass

# Normally: BadConcrete() raises TypeError
# But you can clear __abstractmethods__:
BadConcrete.__abstractmethods__ = frozenset()
obj = BadConcrete()    # works now – but calling area() will fail!

```

```
# This is why ABCs are "interface by convention" not hard enforcement
```

Abstract class with implementation (template method pattern)

```
class DataProcessor(ABC):
    """Template Method Pattern – skeleton algorithm with abstract steps"""

    def process(self, data):
        """Template method – defines the algorithm"""
        data = self.validate(data)    # abstract step
        data = self.transform(data)   # abstract step
        return self.output(data)      # abstract step

    @abstractmethod
    def validate(self, data):
        """Subclass defines validation logic"""
        ...

    @abstractmethod
    def transform(self, data):
        """Subclass defines transformation"""
        ...

    @abstractmethod
    def output(self, data):
        """Subclass defines output format"""
        ...

    def log(self, msg):
        """Concrete shared method"""
        print(f"[{type(self).__name__}] {msg}")

class CSVProcessor(DataProcessor):
    def validate(self, data):
        if not isinstance(data, str):
            raise TypeError("Expected string")
        return data.strip()

    def transform(self, data):
        return [row.split(',') for row in data.split('\n') if row]

    def output(self, data):
        return {'rows': len(data), 'data': data}
```

```
processor = CSVProcessor()
result = processor.process("a,b,c\n1,2,3\n4,5,6")
print(result)
```

ABCs and isinstance — virtual subclassing

```
from abc import ABC, abstractmethod

class Drawable(ABC):
    @abstractmethod
    def draw(self): ...

    @classmethod
    def __subclasshook__(cls, C):
        """Customizes isinstance() and issubclass() checks"""
        if cls is Drawable:
            # ANY class with a 'draw' method is considered a Drawable
            if any("draw" in B.__dict__ for B in C.__mro__):
                return True
            return NotImplemented

class LegacyWidget:
    """Old class – doesn't inherit from Drawable"""
    def draw(self):
        print("drawing widget")

# Virtual subclass check – works because of __subclasshook__
print(isinstance(LegacyWidget(), Drawable)) # True!
print(issubclass(LegacyWidget, Drawable))   # True!

# Register explicitly (alternative to __subclasshook__)
class AnotherDrawable:
    def draw(self): pass

Drawable.register(AnotherDrawable)
print(issubclass(AnotherDrawable, Drawable)) # True
```

Abstraction scenarios

```
# SCENARIO 1: Abstract method calling super
class Animal(ABC):
    @abstractmethod
    def speak(self):
```

```

        return "..."    # abstract methods CAN have implementations!

class Dog(Animal):
    def speak(self):
        parent_sound = super().speak()    # calls abstract method's body!
        return f"Woof! (base: {parent_sound})"

d = Dog()
print(d.speak())    # "Woof! (base: ...)"

# SCENARIO 2: Abstract class __init__
class Plugin(ABC):
    def __init__(self, name):
        self.name = name    # ABC __init__ CAN run via super()

    @abstractmethod
    def execute(self): ...

class MyPlugin(Plugin):
    def __init__(self, name, version):
        super().__init__(name)    # calls ABC's __init__
        self.version = version

    def execute(self):
        print(f"{self.name} v{self.version} executing")

p = MyPlugin("MyPlugin", "1.0")
p.execute()

# SCENARIO 3: Forgetting @property decorator pair
class Base(ABC):
    @property
    @abstractmethod
    def value(self): ...

class Wrong(Base):
    def value(self):    # implemented as METHOD, not property!
        return 42

# Wrong()    # TypeError – value is not a property, so not recognized as implementing
#           # the abstract property

class Correct(Base):
    @property

```



```
def value(self):  
    return 42
```

```
Correct()    # ✓ works
```

3. INHERITANCE

Memory layout in inheritance

```
import sys  
  
class Animal:  
    def __init__(self, name):  
        self.name = name        # stored in instance __dict__  
        self.alive = True  
  
class Dog(Animal):  
    def __init__(self, name, breed):  
        super().__init__(name)  # Animal.__init__ adds name, alive  
        self.breed = breed      # Dog.__init__ adds breed  
  
d = Dog("Rex", "Labrador")  
print(d.__dict__)    # {'name': 'Rex', 'alive': True, 'breed': 'Labrador'}  
# ALL attributes (parent + child) in ONE flat __dict__  
  
# Class hierarchy stored in __mro__:  
print(Dog.__mro__)  
# (<class 'Dog'>, <class 'Animal'>, <class 'object'>)  
  
# Method resolution uses __mro__ chain:  
print(Dog.__dict__)  # only Dog's OWN attributes  
print(Animal.__dict__) # only Animal's OWN attributes  
# Python looks up methods by walking __mro__
```

Single inheritance — complete internal flow

```
class Vehicle:  
    wheels = 4                # class attribute  
  
    def __init__(self, make, speed):  
        self.make = make      # instance attribute  
        self.speed = speed
```

```

def describe(self):
    return f"{self.make} with {self.wheels} wheels at {self.speed}km/h"

def __repr__(self):
    return f"Vehicle({self.make!r}, {self.speed})"

class Car(Vehicle):
    def __init__(self, make, speed, model):
        super().__init__(make, speed) # MUST call or parent attrs missing
        self.model = model

    def describe(self): # OVERRIDES parent method
        base = super().describe() # calls parent's describe
        return f"{base} [{self.model}]"

c = Car("Toyota", 120, "Corolla")
print(c.describe())
# Toyota with 4 wheels at 120km/h [Corolla]

# Attribute lookup chain:
# c.wheels → not in c.__dict__ → not in Car.__dict__
#           → found in Vehicle.__dict__ → 4
# c.make   → found in c.__dict__ → 'Toyota'
# c.describe → not in c.__dict__ → found in Car.__dict__ → Car.describe

```

super() — deep internals

```

# super() is NOT just "call parent class"
# super() returns a PROXY that starts MRO search AFTER a given class

class A:
    def method(self):
        print(f"A.method, self type={type(self).__name__}")

class B(A):
    def method(self):
        print(f"B.method")
        super().method() # next in MRO after B

class C(A):
    def method(self):
        print(f"C.method")
        super().method() # next in MRO after C

```

```

class D(B, C):
    def method(self):
        print(f"D.method")
        super().method()    # next in MRO after D = B

d = D()
d.method()
# D.method
# B.method    (super() in D → B, because MRO is D,B,C,A)
# C.method    (super() in B → C, not A! MRO of the INSTANCE)
# A.method    (super() in C → A)

# Key insight: super() uses the MRO of the INSTANCE (d), not the class
# that defines the method. This enables cooperative multiple inheritance.

# super() with explicit args:
class B(A):
    def method(self):
        super(B, self).method()    # equivalent to super().method() in Python 3
        # super(B, B).method()    # for classmethods – passes class, not instance

```

super() trap — forgetting args in Python 2 style

```

class Parent:
    def __init__(self):
        self.x = 10

class Child(Parent):
    def __init__(self):
        super(Child, self).__init__()    # Python 2 style – works in Python 3
        super().__init__()               # Python 3 style – cleaner

    def method(self):
        # ⚠ TRAP: super() in nested function loses __class__ cell
        def inner():
            super().method()    # NameError! super() needs __class__ cell
                                # __class__ is only set in methods, not nested fns

        # ✅ Fix: capture super() outside
        s = super()

```

```
def inner():
    s.method()          # works – s is captured from enclosing scope
```

Inheritance traps — all scenarios

```
# TRAP 1: Forgetting super().__init__
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        # super().__init__(name) ← forgot this!
        self.breed = breed

d = Dog("Rex", "Lab")
print(d.breed)      # Lab
# print(d.name)     # AttributeError! name was never set

# TRAP 2: Calling wrong super chain
class A:
    def __init__(self): self.a = 1

class B(A):
    def __init__(self):
        A.__init__(self)    # explicit – SKIPS cooperative chain
        self.b = 2

class C(A):
    def __init__(self):
        A.__init__(self)
        self.c = 3

class D(B, C):
    def __init__(self):
        B.__init__(self)    # calls A.__init__ via B
        C.__init__(self)    # calls A.__init__ AGAIN – double init!

# ✅ Fix: use super() everywhere
class D(B, C):
    def __init__(self):
        super().__init__() # cooperative – A.__init__ called once

# TRAP 3: isinstance() with inheritance
```

```

class Animal: pass
class Dog(Animal): pass
class GoldenRetriever(Dog): pass

g = GoldenRetriever()
print(isinstance(g, GoldenRetriever)) # True
print(isinstance(g, Dog))             # True ← IS-A Dog
print(isinstance(g, Animal))          # True ← IS-A Animal
print(isinstance(g, object))          # True ← everything IS-A object

print(type(g) == Dog)                 # False ← exact type check
print(type(g) is GoldenRetriever)     # True ← exact type

# TRAP 4: Class attribute inheritance – shadowing
class Base:
    items = []    # class attribute

class Child(Base):
    pass

Child.items.append(1)    # modifies Base.items!
print(Base.items)       # [1] ← also modified

class Child2(Base):
    items = []    # now Child2 has its OWN items, shadows Base.items

Child2.items.append(2)
print(Base.items)       # [1] ← unchanged
print(Child2.items)     # [2]

# TRAP 5: Method resolution with same name
class A:
    def hello(self): return "A"

class B(A):
    def hello(self): return "B"    # overrides A

class C(A):
    pass                          # inherits A's hello

class D(B, C):
    pass                          # MRO: D, B, C, A

```

```
d = D()
print(d.hello())    # "B" – B comes before C in MRO
```

Inheritance and memory

```
# Each instance has ONE __dict__ containing all attrs (parent + child)
# Class methods are stored in the CLASS's __dict__, not instances

class A:
    class_var = "A"
    def method_a(self): pass

class B(A):
    class_var = "B"    # shadows A's class_var
    def method_b(self): pass

b = B()
b.instance_var = "instance"

print(b.__dict__)      # {'instance_var': 'instance'} – only instance a
                        # ttrs
print(B.__dict__)      # {'class_var': 'B', 'method_b': ...}
print(A.__dict__)      # {'class_var': 'A', 'method_a': ...}

# Memory: method_a lives in A.__dict__
# accessed via b.method_a → not in b.__dict__ → not in B.__dict__
# → found in A.__dict__ → bound method created on the fly
```

4. POLYMORPHISM

What polymorphism really means

```
# Polymorphism = same interface, different implementations
# Python achieves this through:
# 1. Method overriding (runtime polymorphism)
# 2. Duck typing (structural polymorphism)
# 3. Operator overloading (dunder methods)
# 4. Function overloading (via *args/**kwargs or singledispatch)

# Runtime polymorphism – method overriding
class Shape:
    def area(self): raise NotImplementedError
```

```

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h

class Triangle(Shape):
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self): return 0.5 * self.b * self.h

shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]

# Polymorphic call – same interface, different behavior
for shape in shapes:
    print(f"{type(shape).__name__}: area = {shape.area():.2f}")
# Circle: area = 78.50
# Rectangle: area = 24.00
# Triangle: area = 12.00

```

Duck typing — Python's real polymorphism

```

# "If it walks like a duck and quacks like a duck, it's a duck"
# No inheritance required – just implement the right methods

class Duck:
    def quack(self): print("Quack!")
    def walk(self): print("Waddle waddle")

class Person:
    def quack(self): print("I'm quacking like a duck!")
    def walk(self): print("I'm walking like a duck!")

class RubberDuck:
    def quack(self): print("Squeak!")
    def walk(self): print("Rubber ducks don't walk")

def make_it_quack(duck):
    # No isinstance check – works with ANY object that has .quack()
    duck.quack()

make_it_quack(Duck())          # Quack!
make_it_quack(Person())       # I'm quacking like a duck!

```

```

make_it_quack(RubberDuck())    # Squeak!

# ⚠ Duck typing trap: missing method
class Stone:
    pass

# make_it_quack(Stone())    # AttributeError: 'Stone' object has no attribute
                             'quack'

# ✅ Safe duck typing with hasattr
def safe_quack(obj):
    if hasattr(obj, 'quack') and callable(obj.quack):
        obj.quack()
    else:
        print(f"{type(obj).__name__} can't quack!")

```

Operator overloading — polymorphism via dunders

```

class Vector:
    def __init__(self, *components):
        self.components = list(components)

    def __add__(self, other):
        if isinstance(other, Vector):
            if len(self.components) != len(other.components):
                raise ValueError("Vectors must have same dimensions")
            return Vector(*(a+b for a,b in zip(self.components, other.components)))
        return NotImplemented    # let Python try other.__radd__

    def __radd__(self, other):
        return self.__add__(other)

    def __mul__(self, scalar):
        if isinstance(scalar, (int, float)):
            return Vector(*(x * scalar for x in self.components))
        return NotImplemented

    def __rmul__(self, scalar):
        return self.__mul__(scalar)

    def __matmul__(self, other):    # @ operator – dot product
        if isinstance(other, Vector):
            return sum(a*b for a,b in zip(self.components, other.components))

```



```

        return NotImplemented

    def __eq__(self, other):
        if isinstance(other, Vector):
            return self.components == other.components
        return NotImplemented

    def __len__(self):
        return len(self.components)

    def __getitem__(self, idx):
        return self.components[idx]

    def __repr__(self):
        return f"Vector{tuple(self.components)}"

    def __abs__(self):
        # magnitude
        return sum(x**2 for x in self.components) ** 0.5

v1 = Vector(1, 2, 3)
v2 = Vector(4, 5, 6)
print(v1 + v2)      # Vector(5, 7, 9)
print(v1 * 3)       # Vector(3, 6, 9)
print(3 * v1)       # Vector(3, 6, 9) ← __rmul__
print(v1 @ v2)      # 32 (dot product)
print(abs(v1))      # 3.7416...
```

functools.singledispatch — function overloading

```

from functools import singledispatch

@singledispatch
def process(arg):
    """Default implementation"""
    raise TypeError(f"Cannot process {type(arg)}")

@process.register(int)
def _(arg):
    return f"Processing int: {arg * 2}"

@process.register(str)
def _(arg):
    return f"Processing str: {arg.upper()}"
```

```

@process.register(list)
def _(arg):
    return f"Processing list of {len(arg)} items"

@process.register(float)
@process.register(complex)
def _(arg):
    return f"Processing number: {abs(arg)}"

print(process(42))          # Processing int: 84
print(process("hello"))     # Processing str: HELLO
print(process([1, 2, 3]))   # Processing list of 3 items
print(process(3.14))        # Processing number: 3.14
# process({})               # TypeError: Cannot process <class 'dict'>

```

Polymorphism with `__call__` — callable objects

```

class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

class Adder:
    def __init__(self, amount):
        self.amount = amount

    def __call__(self, x):
        return x + self.amount

# Polymorphic – works with any callable (function or callable object)
def apply_transform(value, transform):
    return transform(value)  # transform can be function or callable object

double = Multiplier(2)
add_ten = Adder(10)

print(apply_transform(5, double))      # 10
print(apply_transform(5, add_ten))     # 15
print(apply_transform(5, lambda x: x**2)) # 25 ← lambda also works

```

Polymorphism traps and scenarios

```

# SCENARIO 1: isinstance vs type for polymorphism
class Animal:
    def speak(self): return "..."

class Dog(Animal):
    def speak(self): return "Woof"

class Cat(Animal):
    def speak(self): return "Meow"

def make_speak(animal):
    # ❌ Over-specific – breaks polymorphism
    if type(animal) == Dog:
        print(animal.speak())
    elif type(animal) == Cat:
        print(animal.speak())

    # ✅ Polymorphic – works for all Animal subclasses
    if isinstance(animal, Animal):
        print(animal.speak())

    # ✅ Best – pure duck typing, most flexible
    print(animal.speak()) # just call it – let Python raise if not supported

# SCENARIO 2: Covariant return types
class Builder:
    def set_name(self, name):
        self.name = name
        return self # returns self for chaining

class AdvancedBuilder(Builder):
    def set_version(self, version):
        self.version = version
        return self

# ⚠️ TRAP: chaining breaks with parent methods
b = AdvancedBuilder()
result = b.set_name("test").set_version("1.0") # set_name returns Builder not AdvancedBuilder
# AttributeError: 'Builder' object has no attribute 'set_version'

# ✅ Fix: override in subclass or use generic return type
class Builder:
    def set_name(self, name):

```

```

        self.name = name
        return self    # self is polymorphic – always the actual type!

# self is always the ACTUAL instance – so return self IS covariant!
b = AdvancedBuilder()
result = b.set_name("test")    # returns the AdvancedBuilder instance
print(type(result))            # AdvancedBuilder ← polymorphic!
result.set_version("1.0")      # works!

# SCENARIO 3: Overriding with incompatible signature (LSP violation)
class Stack:
    def push(self, item): ...
    def pop(self): ...

class BoundedStack(Stack):
    def push(self, item, force=False):    # extra argument – ok (backward compatible)
        ...
    def pop(self, default=None):          # extra argument – ok
        ...

class BadStack(Stack):
    def push(self, item, priority):        # REQUIRED extra arg – LSP violation!
        ...                               # callers of Stack.push will break!

# SCENARIO 4: __eq__ and polymorphism
class Point2D:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        if isinstance(other, Point2D):
            return self.x == other.x and self.y == other.y
        return NotImplemented    # ← return NotImplemented, not False!
        # NotImplemented → Python tries other.__eq__(self)
        # False → short-circuits, other.__eq__ never tried

class Point3D(Point2D):
    def __init__(self, x, y, z):
        super().__init__(x, y)
        self.z = z

    def __eq__(self, other):
        if isinstance(other, Point3D):
            return super().__eq__(other) and self.z == other.z

```

```

        return NotImplemented

p2 = Point2D(1, 2)
p3 = Point3D(1, 2, 3)
print(p2 == p3)    # False (Point2D.__eq__ tries, returns NotImplemented)
                   # then Point3D.__eq__ tries, returns NotImplemented
                   # Python falls back to identity: False

```

Complete Memory Management for OOP

Object creation and memory lifecycle

```

import sys, gc

class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

# Memory lifecycle:
n1 = Node(1)          # ref count = 1 (n1)
n2 = Node(2)          # ref count = 1 (n2)

n1.next = n2          # n2 ref count = 2 (n2 + n1.next)
del n2                # n2 ref count = 1 (still n1.next)
del n1                # n1 ref count = 0 → immediately freed
                     # n1's cleanup decrements n2's count → 0 → n2 freed

```

Circular references — where ref counting fails

```

import gc

class Node:
    def __init__(self, val):
        self.val = val
        self.ref = None

# Create circular reference:
a = Node(1)
b = Node(2)
a.ref = b    # a → b
b.ref = a    # b → a ← circular!

```

```

del a
del b
# Both still have ref count = 1 (from each other)
# Neither gets freed by ref counting!

# Python's cyclic GC handles this:
gc.collect() # explicitly trigger cyclic GC
# Both freed now

# Check if cyclic GC found them:
print(gc.collect()) # returns number of objects collected

# Weak references – break cycles without preventing cleanup
import weakref

class Cache:
    def __init__(self):
        self.data = {}

    def store(self, key, obj):
        self.data[key] = weakref.ref(obj) # weak ref – doesn't increment count

    def retrieve(self, key):
        ref = self.data.get(key)
        if ref is None: return None
        obj = ref() # dereference: returns obj or None if collected
        return obj

cache = Cache()
class BigObject:
    def __init__(self, data): self.data = data

obj = BigObject("important data")
cache.store("key", obj)

print(cache.retrieve("key").data) # "important data"
del obj # ref count → 0, immediately freed
print(cache.retrieve("key")) # None – weakref returns None after collection

```

`__del__` and memory — the destructor trap

```

import gc

class Resource:
    def __init__(self, name):
        self.name = name
        print(f"Acquired: {name}")

    def __del__(self):
        # ⚠ NOT guaranteed to run at specific time
        # ⚠ NOT called if circular refs exist (until cyclic GC)
        # ⚠ Exception in __del__ is silently ignored
        # ⚠ Global variables may be None during interpreter shutdown
        print(f"Released: {self.name}")

r = Resource("DB Connection")
del r  # __del__ called immediately in CPython (no cycles)
# Released: DB Connection

# ⚠ TRAP: __del__ with circular reference
class Bad:
    def __init__(self):
        self.self_ref = self  # circular!

    def __del__(self):
        print("cleaning up")

b = Bad()
del b  # NOT immediately cleaned up – cycle!
gc.collect()  # now __del__ is called

# ✅ BETTER: use context manager instead of __del__
class SafeResource:
    def __enter__(self):
        print("Acquired")
        return self

    def __exit__(self, *args):
        print("Released")  # GUARANTEED to run when with block exits

with SafeResource() as r:
    pass  # Acquired → Released

```

Memory profiling OOP

```

import sys
import tracemalloc

tracemalloc.start()

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class PointSlots:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

# Compare memory usage
points_dict = [Point(i, i) for i in range(100000)]
snapshot1 = tracemalloc.take_snapshot()

points_slots = [PointSlots(i, i) for i in range(100000)]
snapshot2 = tracemalloc.take_snapshot()

# sys.getsizeof comparison
p1 = Point(1, 2)
p2 = PointSlots(1, 2)

print(f"With __dict__: {sys.getsizeof(p1) + sys.getsizeof(p1.__dict__)} bytes")
# With __dict__: 152 bytes

print(f"With __slots__: {sys.getsizeof(p2)} bytes")
# With __slots__: 56 bytes
# ~3x more memory efficient!

```

Master Scenarios — Interview Level

Scenario 1 — MRO + super + init

```

class A:
    def __init__(self):
        print("A")
        super().__init__()

```



```

class B(A):
    def __init__(self):
        print("B")
        super().__init__()

class C(A):
    def __init__(self):
        print("C")
        super().__init__()

class D(B, C):
    def __init__(self):
        print("D")
        super().__init__()

D()
# D → B → C → A (C3 MRO)
# Prints: D, B, C, A

```

Scenario 2 — Property + inheritance

```

class Base:
    @property
    def value(self):
        return 42

class Child(Base):
    @property
    def value(self):          # overrides getter only
        return super().value * 2 # 84

    # ⚠ TRAP: to add setter in child, must redefine getter too!
    @value.setter
    def value(self, v):
        self._value = v

c = Child()
print(c.value)    # 84
c.value = 10
print(c.value)    # 84 (getter doesn't use _value – still returns super * 2)

```

Scenario 3 — Polymorphism + abstract + memory

```

from abc import ABC, abstractmethod
import sys

class Animal(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def speak(self): ...

    def __repr__(self):
        return f"{type(self).__name__}({self.name!r})"

class Dog(Animal):
    def speak(self): return "Woof"

class Cat(Animal):
    def speak(self): return "Meow"

animals = [Dog("Rex"), Cat("Whiskers"), Dog("Buddy")]

# Polymorphic iteration
sounds = [a.speak() for a in animals]    # ['Woof', 'Meow', 'Woof']

# Memory – each instance shares methods via class, not instance
print(Dog.speak is Cat.speak)    # False – different functions
print(Dog("a").speak == Dog("b").speak)    # False – bound to different instances

# But class methods are shared:
d1, d2 = Dog("a"), Dog("b")
print(d1.speak.__func__ is d2.speak.__func__)    # True – same function object!

```

Scenario 4 — Encapsulation bypass

```

class Secret:
    def __init__(self):
        self.__data = "classified"

s = Secret()

# All these access the "private" attribute:
print(s._Secret__data)    # direct mangled access

```

```

print(vars(s)['_Secret__data'])# via vars()
print(s.__dict__['_Secret__data']) # via __dict__

# Modify it:
s._Secret__data = "hacked"
print(s._Secret__data)    # "hacked"

```

Complete Cheat Sheet

ENCAPSULATION

```

public    → self.x          (access anywhere)
protected → self._x         (convention only, still accessible)
private   → self.__x         (mangled to _Class__x, not truly private)
@property → getter/setter with validation
__slots__ → 3x memory savings, blocks new attributes

```

ABSTRACTION

```

ABC + @abstractmethod → enforces interface
__abstractmethods__   → frozenset of unimplemented methods
instantiation blocked → TypeError if any abstract method missing
__subclasshook__      → customize isinstance/issubclass

```

INHERITANCE

```

Single:  Child(Parent)
Multi:   Child(A, B, C)
MRO:     C3 linearization – always left-to-right, parent after children
super()  → proxy, next in MRO of INSTANCE, not the defining class
__dict__ → flat, contains ALL attrs (parent + child combined)
Method   → found by walking __mro__, first match wins

```

POLYMORPHISM

```

Override    → same name, different behavior
Duck typing  → interface by capability, not by type
Overload     → singledispatch or *args/**kwargs
Operator     → __add__, __mul__, __eq__, etc.
NotImplemented → return from dunder to let Python try reflected op

```

MEMORY

```

ref count → primary GC (immediate dealloc on count=0)
cyclic GC  → handles circular references
__del__    → NOT reliable, use context managers instead
weakref    → reference without incrementing count

```

```
__slots__ → eliminates __dict__, ~3x memory savings  
tracemalloc → profiling tool for object memory usage
```

What the Task Wants

The task is asking you to learn **database management using Python's SQLAlchemy library**. Specifically, it's split into 2 scripts:

- **Script 1 (db_setup.py)** — Create a SQLite database called `films_db` with a table called `films`
- **Script 2 (db_operations.py)** — Perform CRUD operations (Create, Read, Update, Delete) on that table

Think of it like building a mini movie catalog app backed by a real database.

db_setup.py — Line by Line

```
from sqlalchemy import create_engine, Column, Integer, String
```

Imports the tools needed: `create_engine` connects to a DB, `Column/Integer/String` define table columns.

```
from sqlalchemy.orm import declarative_base
```

Imports `declarative_base` — this is the **parent class** that lets you define database tables as Python classes.

```
engine = create_engine("sqlite:///films_db.db", echo=True)
```

Creates a connection to a SQLite database file called `films_db.db`. `echo=True` means SQLAlchemy will **print every SQL query** it runs (useful for learning/debugging).

```
Base = declarative_base()
```

Creates a base class. Every table model you define must **inherit from this Base**.

```
class Film(Base):  
    __tablename__ = "films"
```

Defines a Python class that maps to the `films` table in the database. SQLAlchemy will translate this class into an actual SQL table.

```
id = Column(Integer, primary_key=True)  
title = Column(String, nullable=False)  
director = Column(String, nullable=False)  
release_year = Column(Integer, nullable=False)
```

Defines the 4 columns of the table. `primary_key=True` means `id` is the unique identifier. `nullable=False` means the field **cannot be empty**.

```
def __repr__(self):
    return f"<Film(id={self.id}, title='{self.title}', ...)>"
```

A special method that controls how a Film object **looks when printed**. Without this, printing a film would just show a memory address.

```
if __name__ == "__main__":
    Base.metadata.create_all(engine)
    print("Database and table created successfully.")
```

`create_all(engine)` looks at all classes inheriting from `Base` and **creates their tables in the database**. The `if __name__` block ensures this only runs when you execute this file directly.

db_operations.py — Line by Line

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from db_setup import Film, Base
```

Imports `sessionmaker` (for DB sessions) and imports the `Film` model from the setup file so both scripts share the same table definition.

```
engine = create_engine("sqlite:///films_db.db", echo=True)
Session = sessionmaker(bind=engine)
session = Session()
```

Reconnects to the **same database file**. A **session** is like a conversation with the database — all your operations go through it before being committed (saved).

+ Add 3 Films

```
film1 = Film(title="Inception", director="Christopher Nolan", release_year=2010)
film2 = Film(title="Interstellar", director="Christopher Nolan", release_year=2014)
film3 = Film(title="The Matrix", director="Lana Wachowski & Lilly Wachowski", release_year=1999)
```

Creates 3 Python objects — **not yet saved to DB**, just in memory.

```
session.add_all([film1, film2, film3])
session.commit()
```

`add_all` stages all 3 films. `commit()` **permanently writes** them to the database (like hitting Save).

Update 1 Film

```
film_to_update = session.query(Film).filter_by(title="The Matrix").first()
```

Queries the DB: *"Find the first film where title = 'The Matrix'".* This is equivalent to SQL: `SELECT * FROM films WHERE title='The Matrix' LIMIT 1`.

```
if film_to_update:
    film_to_update.release_year = 2000
    session.commit()
```

If found, change its `release_year` to 2000 and commit. SQLAlchemy **detects the change** automatically and runs an `UPDATE` query.

Print All Films

```
films = session.query(Film).all()
for film in films:
    print(film)
```

`query(Film).all()` fetches every row from the films table. The loop prints each one using the `__repr__` method defined earlier.

Delete All Films

```
session.query(Film).delete()
session.commit()
```

Deletes **every row** from the films table. Equivalent to SQL: `DELETE FROM films`.

```
session.close()
```

Closes the connection to the database cleanly. Always good practice.

The Big Picture

Concept	What it does
<code>engine</code>	The connection to your DB file
<code>Base</code>	Blueprint system for defining tables

Concept	What it does
<code>Film(Base)</code>	A Python class = a database table
<code>session</code>	Your workspace for all DB operations
<code>commit()</code>	Saves changes permanently
<code>query()</code>	Reads data from the DB